

Physique dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Calcul Vectoriel	3
Session 2 : Cinématique 2D et Mouvement de Projectile	10
Position en 2D	11
Vitesse en 2D	12
Accélération en 2D	14
Session 3 : Les lois de Newton – La méthode physique	17
Objectifs de la session	17
La démarche physique : modéliser l'univers	17
Session 4 : Les forces de la nature	19
Objectifs de la session	19
Session 5 : Intégration numérique 101 - Méthode d'Euler	21
Objectifs de la session	21
Session 6 : L'impulsion, les collisions	29
Objectifs de la session	29
Session 7 : Broad et Narrow phases – Part 1	33
Objectifs de la session	33
Session 8 : Intégration de Verlet et Position Based Dynamics	42
Objectifs de la session	42
Session 9 : TP - Particules	49
Objectifs du TP	49
Session 10 : Introduction à la physique des rotations	51
Objectifs de la session	51
Étape 1 : Classe RigidBody	52
Étape 2 : ApplyForce(force, worldPoint)	52
Étape 3 : Intégrateur	52
Session 11 : Broad et Narrow phases – Part 2	53
Objectifs de la session	53
Session 12 : TP - Les planètes - Runge-Kutta	55
Objectifs du TP	55
Session 13 : Les moteurs physiques commerciaux	57
Objectifs de la session	57
Session 14 : Corps rigides – Boite de collision	59
Objectifs de la session	59
Session 15 : Joints, Moteurs	61
Objectifs de la session	61
Session 16 : Les contraintes – et l'IK	63
Objectifs de la session	63
Session 17 : TP - Véhicules	66
Objectifs du TP	66
Session 18 : Personnages	68
Objectifs de la session	68
Session 19 : Physique, GPU et IA	70

Objectifs de la session	70
Session 20 : Résumé - Quiz final	72
Objectifs de la session	72
Concepts clés à retenir	72
Quiz final	72

Session 1 : Géométrie et Calcul Vectoriel

Définitions

Un vecteur est une grandeur ayant une direction et une magnitude.

On dit qu'il est de dimension n si il a n composantes.

Dans le plan, $n = 2$, donc un vecteur a 2 composantes, 2 dimensions.

🔗 Représentation

Un vecteur est représenté par une flèche. Il peut être placé n'importe où dans le plan. On le place habituellement par rapport au contexte de ce qu'il représente.



Fig. 1. – Un vecteur

💡 Notation vectorielle

Même si le vecteur est défini par ses magnitude et direction, on utilise une notation vectorielle, qui décompose le vecteur selon les vecteurs $\hat{i}$ et $\hat{j}$ du plan cartésien. On retrouve ses 2 dimensions représentées en colonne.

Exemple.

$$\vec{v} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2\hat{i} + 3\hat{j}$$

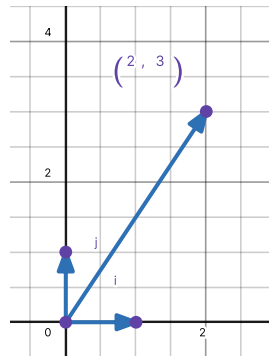


Fig. 2. – Le vecteur $\vec{v}$

🔗 Déplacement

Un déplacement peut se décrire par le vecteur reliant le point A (départ) au point B (arrivée) :

$$\overrightarrow{AB} = \begin{pmatrix} x_B - x_A \\ y_B - y_A \end{pmatrix}$$

🔗 Des vecteurs célèbres en physique

- **Le vecteur position**

Souvent noté $\vec{r}$ (radius), représente la position d'un point dans l'espace en partant de l'origine.

- **Le vecteur vitesse**

Noté $\vec{v}$, représente la vitesse d'un objet, souvent représenté partant du centre de l'objet.

- **Le vecteur accélération**

Noté $\vec{a}$, représente l'accélération d'un objet, souvent représenté partant du centre de l'objet.

Magnitude et direction

La longueur du vecteur, aussi appelée magnitude, est notée $\|\vec{v}\|$, et parfois simplement $|\vec{v}|$.

En 2D : On utilise le théorème de Pythagore pour calculer la norme (autre nom de la magnitude) :

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2}$$

Vecteur Unitaire ($\hat{u}$) : Vecteur de longueur 1 direction $\vec{v}$:

$$\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$$

Un vecteur unitaire est un vecteur de longueur 1, il est souvent utilisé pour représenter une direction sans avoir à se soucier de la magnitude.

Opérations de Base

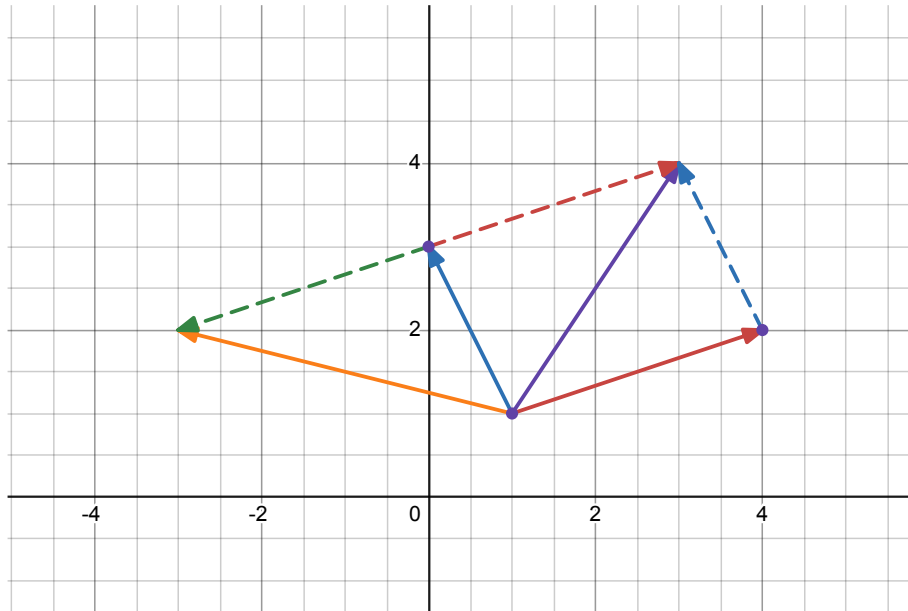
Soit $\vec{u} = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$.

Addition :

$$\vec{u} + \vec{v} = \begin{pmatrix} u_x + v_x \\ u_y + v_y \end{pmatrix}$$

Soustraction :

$$\vec{u} - \vec{v} = \begin{pmatrix} u_x - v_x \\ u_y - v_y \end{pmatrix}$$

Fig. 3. – Le vecteur $\vec{u}$ (rouge) et le vecteur $\vec{v}$ (bleu)

Multiplication par scalaire (k) :

$$k \cdot \vec{u} = \begin{pmatrix} ku_x \\ ku_y \end{pmatrix}$$

Produit Scalaire

Résultat : un **nombre** (scalaire).

Formule algébrique :

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y + u_z v_z$$

Formule géométrique :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \cdot \|\vec{v}\| \cdot \cos(\theta)$$

1. Si $\vec{u} \cdot \vec{v} = 0$, alors $\vec{u} \perp \vec{v}$.
2. Si $\vec{u} \cdot \vec{v} > 0$, alors l'angle entre les vecteurs est inférieur à 90° . (devant)
3. Si $\vec{u} \cdot \vec{v} < 0$, alors l'angle entre les vecteurs est supérieur à 90° . (derrière)

Angle entre 2 vecteurs

On isole $\cos(\theta)$. :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}\right)$$

Projections

Projection de $\vec{u}$ sur $\vec{v}$. On utilise le produit scalaire, dans la direction sur laquelle on projette.

$$\text{proj}_{\vec{v}}(\vec{u}) = \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{v}\|^2} \right) \cdot \vec{v}$$

Et si $\vec{v}$ est normalisé (direction de la droite sur laquelle on projète), on peut simplifier :

$$\text{proj}_{\vec{v}}(\vec{u}) = (\vec{u} \cdot \vec{v}) \cdot \vec{v}$$

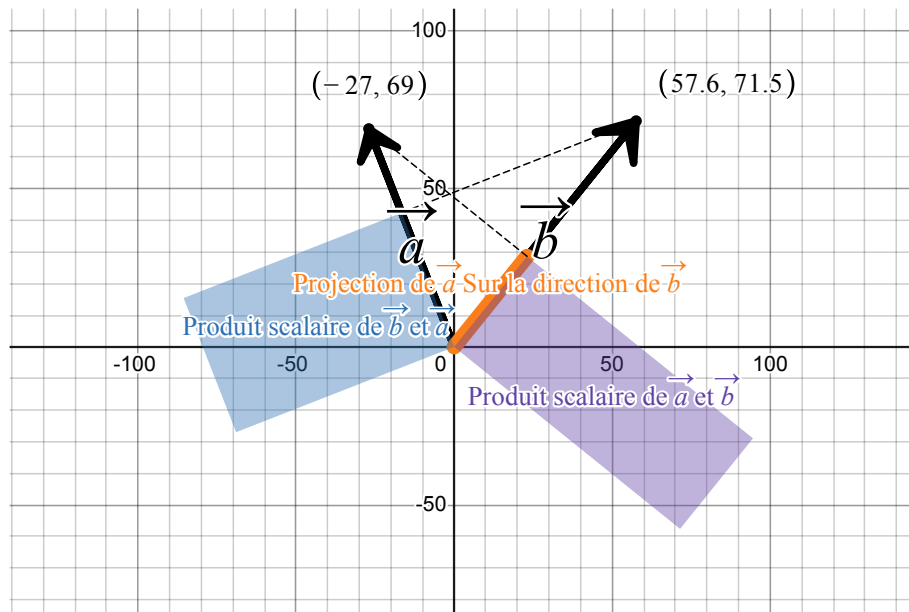


Fig. 4. – Produit scalaire, projection

Produit Vectoriel (3D)

Résultat : un **vecteur** perpendiculaire.

$$\vec{w} = \vec{a} \times \vec{b}$$

Commutativité : Attention, le produit vectoriel n'est pas commutatif ! On a un changement de signe :

$$\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$$

Magnitude :

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \sin(\theta)$$

Direction : Le produit vectoriel est perpendiculaire aux deux vecteurs :

$$\vec{a} \times \vec{b} \perp \vec{a}$$

$$\vec{a} \times \vec{b} \perp \vec{b}$$

On utilise la règle de la main droite pour trouver la direction du produit vectoriel.

- Le pouce pointe sur a

- l'index sur b
- le majeur sur $a \times b$.

Calcul (Déterminant) : Le déterminant est un outil algébrique qui permet de trouver les solutions d'un problème de n équations avec n inconnues. Cette solution représente, en quelque sorte une direction émergente (perpendiculaire) des équations.

$$a = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \text{ et } b = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}$$

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Exercice : Le parallélogramme

Soit deux vecteurs dans l'espace 3D :

$$\vec{a} = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} \text{ et } \vec{b} = \begin{pmatrix} 0 \\ 3 \\ 2 \end{pmatrix}$$

1. Calculez le produit vectoriel $\vec{w} = \vec{a} \times \vec{b}$ en utilisant la méthode du déterminant.
2. Vérifiez que $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et à $\vec{b}$ à l'aide du produit scalaire.
3. Calculez l'aire du parallélogramme formé par $\vec{a}$ et $\vec{b}$.

Solution : Produit vectoriel par le déterminant. On pose la matrice avec $\hat{i}, \hat{j}, \hat{k}$ en première ligne :

$$\vec{a} \times \vec{b} = \begin{pmatrix} \hat{i} & \hat{j} & \hat{k} \\ 2 & 1 & 0 \\ 0 & 3 & 2 \end{pmatrix}$$

Développement par la première ligne (cofacteurs) :

$$\vec{w} = \hat{i} \cdot \begin{pmatrix} 1 & 0 \\ 3 & 2 \end{pmatrix} - \hat{j} \cdot \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} + \hat{k} \cdot \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Calcul des sous-déterminants 2x2 :

$$\vec{w} = \hat{i} \cdot (1 \cdot 2 - 0 \cdot 3) - \hat{j} \cdot (2 \cdot 2 - 0 \cdot 0) + \hat{k} \cdot (2 \cdot 3 - 1 \cdot 0)$$

$$\vec{w} = \hat{i} \cdot (2) - \hat{j} \cdot (4) + \hat{k} \cdot (6)$$

$$\vec{w} = \begin{pmatrix} 2 \\ -4 \\ 6 \end{pmatrix}$$

Solution : Vérification de perpendicularité.

$$\vec{w} \cdot \vec{a} = (2)(2) + (-4)(1) + (6)(0) = 4 - 4 + 0 = 0 \quad \text{checkmark}$$

$$\vec{w} \cdot \vec{b} = (2)(0) + (-4)(3) + (6)(2) = 0 - 12 + 12 = 0 \quad \text{checkmark}$$

Les deux produits scalaires sont nuls : $\vec{w}$ est bien perpendiculaire à $\vec{a}$ et $\vec{b}$.

Solution : Aire du parallélogramme. L'aire du parallélogramme formé par deux vecteurs est égale à la norme de leur produit vectoriel :

$$\text{Aire} = \|\vec{w}\| = \sqrt{2^2 + (-4)^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} \approx 7.48$$

Exercice : Le food truck

Un food truck vend trois types de repas : des burgers à 8, des tacos à 5, et des salades à 6.

Samedi, il a vendu 100 repas pour un total de 620 dollars.

On sait de plus que le nombre de tacos vendus est égal au nombre de burgers plus le double du nombre de salades.

Combien de chaque repas a-t-il vendu ?

Mise en équation :

Soit x le nombre de burgers, y le nombre de tacos, z le nombre de salades.

1. « Total des repas » : $x + y + z = 100$
2. « Total des ventes » : $8x + 5y + 6z = 620$
3. « Relation tacos » : $y = x + 2z$, soit $-x + y - 2z = 0$

Système :

$$x + y + z = 100$$

$$8x + 5y + 6z = 620$$

$$-x + y - 2z = 0$$

Solution : Règle de Cramer. Si $\Delta \neq 0$, alors $x = \frac{\Delta_x}{\Delta}$, $y = \frac{\Delta_y}{\Delta}$, $z = \frac{\Delta_z}{\Delta}$.

Déterminant principal Δ :

$$\Delta = \begin{pmatrix} 1 & 1 & 1 \\ 8 & 5 & 6 \\ -1 & 1 & -2 \end{pmatrix}$$

Développement par la première ligne :

$$\Delta = 1 \cdot \begin{pmatrix} 5 & 6 \\ 1 & -2 \end{pmatrix} - 1 \cdot \begin{pmatrix} 8 & 6 \\ -1 & -2 \end{pmatrix} + 1 \cdot \begin{pmatrix} 8 & 5 \\ -1 & 1 \end{pmatrix}$$

$$\Delta = 1 \cdot (-10 - 6) - 1 \cdot (-16 + 6) + 1 \cdot (8 + 5)$$

$$\Delta = -16 - (-10) + 13 = -16 + 10 + 13 = 7$$

Solution : Calcul de Δ_x , Δ_y , Δ_z . Δ_x (remplacer la colonne 1 par les constantes) :

$$\Delta_x = \begin{pmatrix} 100 & 1 & 1 \\ 620 & 5 & 6 \\ 0 & 1 & -2 \end{pmatrix}$$

$$\Delta_x = 100 \cdot (-16) - 1 \cdot (-1240) + 1 \cdot (620) = -1600 + 1240 + 620 = 260$$

Δ_y (remplacer la colonne 2 par les constantes) :

$$\Delta_y = \begin{pmatrix} 1 & 100 & 1 \\ 8 & 620 & 6 \\ -1 & 0 & -2 \end{pmatrix}$$

$$\Delta_y = 1 \cdot (-1240) - 100 \cdot (-10) + 1 \cdot (620) = -1240 + 1000 + 620 = 380$$

Δ_z (remplacer la colonne 3 par les constantes) :

$$\Delta_z = \begin{pmatrix} 1 & 1 & 100 \\ 8 & 5 & 620 \\ -1 & 1 & 0 \end{pmatrix}$$

$$\Delta_z = 1 \cdot (-620) - 1 \cdot (620) + 100 \cdot (13) = -620 - 620 + 1300 = 60$$

Solution : Résultats et vérification.

$$x = \frac{\Delta_x}{\Delta} = \frac{260}{7} \approx 37 \text{ burgers}$$

$$y = \frac{\Delta_y}{\Delta} = \frac{380}{7} \approx 54 \text{ tacos}$$

$$z = \frac{\Delta_z}{\Delta} = \frac{60}{7} \approx 9 \text{ salades}$$

Vérification :

1. $37 + 54 + 9 = 100$ checkmark
2. $8(37) + 5(54) + 6(9) = 296 + 270 + 54 = 620$ checkmark
3. $54 = 37 + 2(9) = 37 + 18 = 55 \approx 54$ checkmark (arrondi)

Note

Les valeurs exactes sont $x = \frac{260}{7}$, $y = \frac{380}{7}$, $z = \frac{60}{7}$. Dans un cas réel, on ajusterait les données pour obtenir des nombres entiers. L'important ici est la méthode.

Session 2 : Cinématique 2D et Mouvement de Projectile

🔗 Introduction à la cinématique

La cinématique est la branche de la mécanique qui décrit le mouvement des objets sans considérer les causes du mouvement (les forces). Contrairement à la dynamique, qui relie le mouvement aux forces.

Notre objectif dans ce bloc est de développer les outils mathématiques pour décrire précisément *comment* les objets se déplacent.

Position, Vitesse et Accélération

La position, la vitesse et l'accélération sont des concepts qui permettent de décrire précisément comment un objet se comporte.

- La **position** est le vecteur qui indique la localisation d'un point dans l'espace.
- La **vitesse** est le vecteur qui indique la vitesse du point, définie comme la variation (dérivée) de la position par rapport au temps.
- L'**accélération** est le vecteur qui indique l'accélération du point, définie comme la variation (dérivée) de la vitesse par rapport au temps.

🔗 Cas particuliers

- Lorsque la position est constante, le point est au repos, immobile. Sa vitesse et son accélération sont nulles.
- Lorsque la vitesse est constante, le point se déplace à vitesse constante en ligne droite. Son accélération est nulle.
- Lorsque l'accélération est constante, la vitesse augmente constamment, le point se déplace de plus en plus vite.

1. Introduction et Rappels

Rappels sur les vecteurs

Rappel de la définition d'un vecteur comme une quantité possédant une magnitude (longueur, norme) et une direction.

Représentation d'un vecteur en 2D à l'aide de ses composantes :

$$\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

.

Opérations vectorielles de base :

$$\vec{a} + \vec{b} = \begin{pmatrix} a_x + b_x \\ a_y + b_y \end{pmatrix}$$

$$\vec{a} - \vec{b} = \begin{pmatrix} a_x - b_x \\ a_y - b_y \end{pmatrix}$$

Multiplication par un scalaire :

$$k\vec{a} = \begin{pmatrix} ka_x \\ ka_y \end{pmatrix}$$

Position en 2D

Vecteur Position

Pour décrire la localisation d'un point dans un plan bidimensionnel, nous utilisons le **vecteur position**, noté $\vec{r}$ (ou parfois $\vec{s}$ ou $\vec{x}$).

Avec un système de coordonnées cartésiennes (x, y) , le vecteur position d'un point P de coordonnées (x, y) est :

$$\vec{r} = \begin{pmatrix} x \\ y \end{pmatrix} = x\hat{i} + y\hat{j}$$

où $\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ et $\hat{j} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ sont les vecteurs unitaires.

- Le vecteur position pointe de l'origine vers la position de l'objet.
- La **magnitude** $|\vec{r}| = \sqrt{x^2 + y^2}$ représente la distance à l'origine.
- La **direction** est donnée par l'angle θ avec $\tan(\theta) = \frac{y}{x}$.

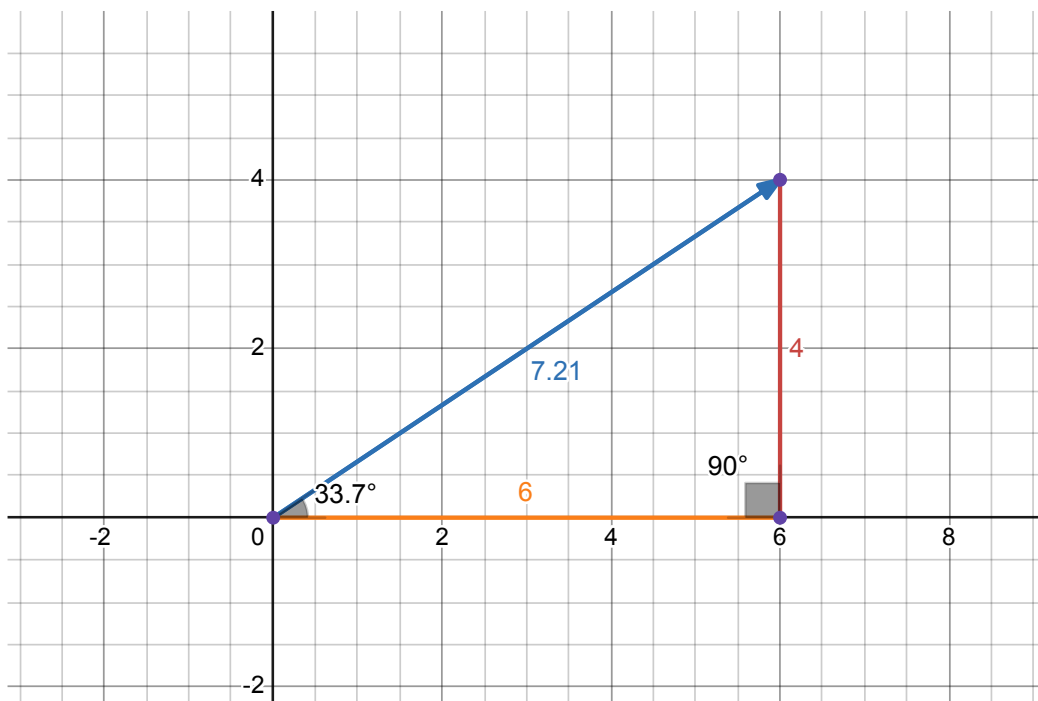


Fig. 5. – Plan 2D avec vecteurs position et vitesse

💡 Trajectoire

Si la position d'un objet change au cours du temps, on décrit son mouvement par :

$$\vec{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

L'ensemble des points atteints forme la **trajectoire** — une courbe dans l'espace. $x(t)$ et $y(t)$ sont des fonctions du temps.

Exemples de trajectoires :

• **Mouvement rectiligne uniforme :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t \end{pmatrix}$$

• **Mouvement circulaire uniforme :**

$$\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$$

• **Mouvement parabolique (projectile) :**

$$\vec{r}(t) = \begin{pmatrix} x_0 + v_{0x}t \\ y_0 + v_{0y}t - \frac{1}{2}gt^2 \end{pmatrix}$$

Vitesse en 2D

Vitesse Moyenne

Le **déplacement** d'un objet pendant un intervalle $\Delta t = t_f - t_i$ est :

$$\Delta \vec{r} = \vec{r}_f - \vec{r}_i = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

Le **vecteur vitesse moyenne** $\vec{v}_{\text{moy}}$ est le rapport du déplacement au temps écoulé :

$$\vec{v}_{\text{moy}} = \frac{\Delta \vec{r}}{\Delta t} = \begin{pmatrix} \frac{\Delta x}{\Delta t} \\ \frac{\Delta y}{\Delta t} \end{pmatrix} = \begin{pmatrix} v_{\text{moy},x} \\ v_{\text{moy},y} \end{pmatrix}$$

La vitesse moyenne a la même direction que le déplacement, et sa magnitude est le déplacement total divisé par le temps écoulé.

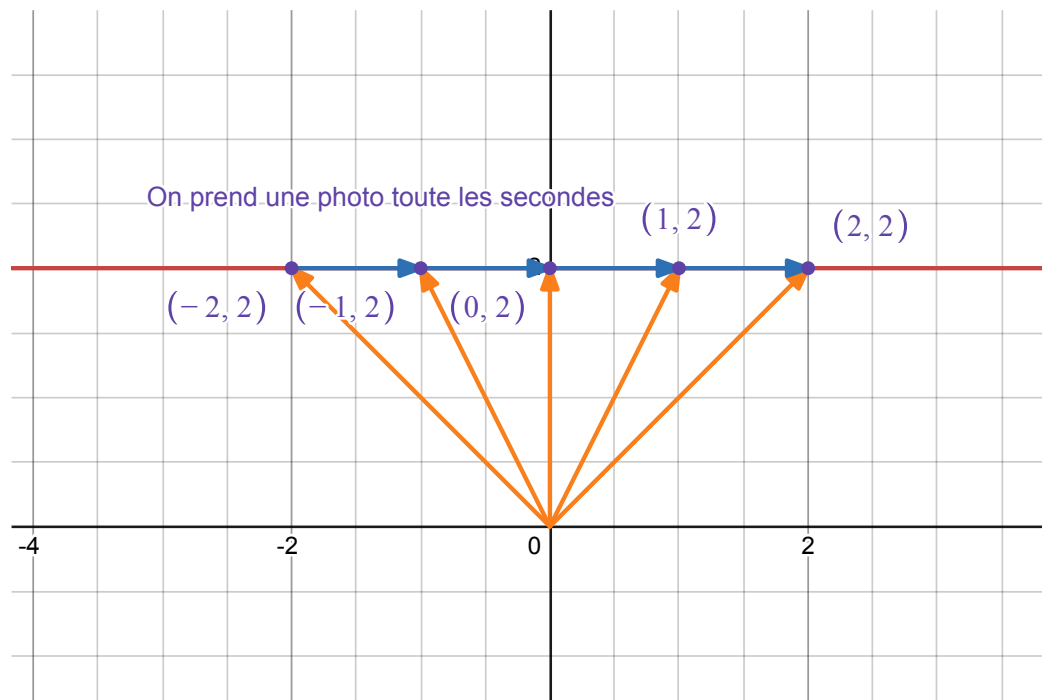


Fig. 6. – Plan 2D avec vecteurs position et vitesse

Exemple. Si une voiture se déplace de $\vec{r}_i = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ à $\vec{r}_f = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ en $\Delta t = 5$ secondes, alors le déplacement est $\Delta \vec{r} = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ et la vitesse moyenne est $\vec{v}_{\text{moy}} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ m/s.

Vitesse Instantanée

La **vitesse instantanée** $\vec{v}(t)$ est la limite de la vitesse moyenne lorsque $\Delta t \rightarrow 0$:

$$\vec{v}(t) = \lim_{\Delta t \rightarrow 0} \frac{\Delta \vec{r}}{\Delta t} = \frac{d\vec{r}}{dt}$$

En composantes :

$$\vec{v}(t) = \begin{pmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{pmatrix} = \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix}$$

- La **magnitude** $|\vec{v}(t)| = \sqrt{v_{x(t)}^2 + v_{y(t)}^2}$ est la **vitesse scalaire**.
- La **direction** est tangente à la trajectoire au point considéré.

🔗 Application aux types de trajectoire

1. Mouvement rectiligne uniforme :

$$\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

— Le vecteur vitesse est **constant**.

2. Mouvement circulaire uniforme :

$$\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$$

Magnitude constante : $|\vec{v}| = R\omega$. La vitesse est tangente au cercle.

3. Mouvement parabolique (projectile) :

$$\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - gt \end{pmatrix}$$

La composante horizontale v_x reste constante, la verticale v_y diminue linéairement.

Accélération en 2D

Accélération Moyenne

Le **vecteur accélération moyenne** $\vec{a}_{\text{moy}}$ pendant un intervalle $\Delta t = t_f - t_i$ est le rapport du changement de vitesse au temps écoulé :

$$\vec{a}_{\text{moy}} = \frac{\Delta \vec{v}}{\Delta t} = \begin{pmatrix} \frac{\Delta v_x}{\Delta t} \\ \frac{\Delta v_y}{\Delta t} \end{pmatrix} = \begin{pmatrix} a_{\text{moy},x} \\ a_{\text{moy},y} \end{pmatrix}$$

L'accélération moyenne a la direction du changement de vitesse.

Accélération Instantanée

L'**accélération instantanée** $\vec{a}(t)$ est la dérivée de la vitesse par rapport au temps :

$$\vec{a}(t) = \frac{d\vec{v}}{dt} = \begin{pmatrix} \frac{dv_x(t)}{dt} \\ \frac{dv_y(t)}{dt} \end{pmatrix} = \begin{pmatrix} \frac{d^2x(t)}{dt^2} \\ \frac{d^2y(t)}{dt^2} \end{pmatrix}$$

L'accélération peut changer la magnitude de la vitesse (accélérer/décélérer), sa direction, ou les deux.

Cas particulier : Accélération constante

Si l'accélération $\vec{a}$ est constante, on peut **intégrer** pour obtenir la vitesse et la position :

$$\vec{v}(t) = \vec{v}_0 + \vec{a}t = \begin{pmatrix} v_{0x} + a_x t \\ v_{0y} + a_y t \end{pmatrix}$$

$$\vec{r}(t) = \vec{r}_0 + \vec{v}_0 t + \frac{1}{2} \vec{a} t^2 = \begin{pmatrix} x_0 + v_{0x} t + \frac{1}{2} a_x t^2 \\ y_0 + v_{0y} t + \frac{1}{2} a_y t^2 \end{pmatrix}$$

où $\vec{v}_0 = \begin{pmatrix} v_{0x} \\ v_{0y} \end{pmatrix}$ est la vitesse initiale et $\vec{r}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ est la position initiale.

C'est ce cas particulier qui est crucial pour l'étude du mouvement de projectile sous l'effet de la gravité.

🔗 Exemples par type de trajectoire

1. Mouvement rectiligne uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} x_0 + v_x t \\ y_0 + v_y t \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

La dérivée d'un vecteur constant est nulle. Aucune accélération n'est nécessaire pour maintenir le mouvement rectiligne uniforme.

2. Mouvement circulaire uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \end{pmatrix}$

On observe que $\vec{a}(t) = -\omega^2 \vec{r}(t)$: l'accélération est dirigée vers le centre du cercle.

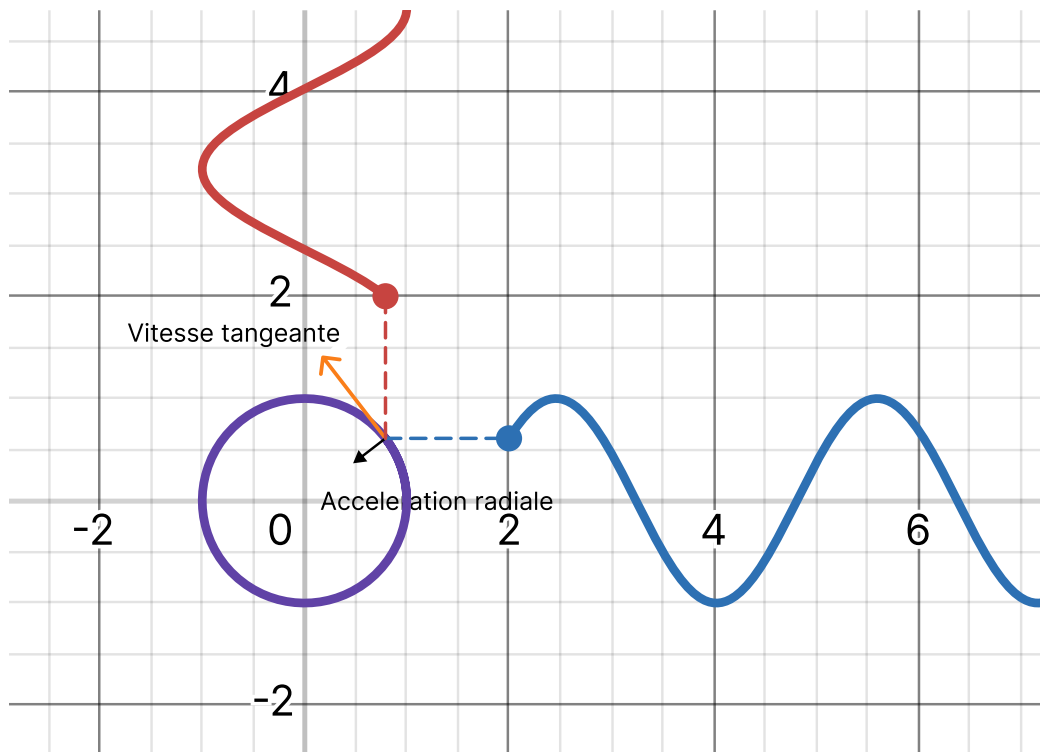


Fig. 7. – Le mouvement circulaire uniforme

3. Mouvement parabolique (projectile) :

- Position : $\vec{r}(t) = \begin{pmatrix} v_{0x} t \\ y_0 + v_{0y} t - \frac{1}{2} g t^2 \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - g t \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ -g \end{pmatrix}$

L'accélération est **constante** et dirigée vers le bas. Avec $g = 9.81 \text{ m/s}^2$, $\vec{a} = \begin{pmatrix} 0 \\ -9.81 \end{pmatrix} \text{ m/s}^2$. Seule la composante verticale de la vitesse change.

Session 3 : Les lois de Newton – La méthode physique

Objectifs de la session

- Réviser la cinématique vue précédemment.
- Introduire les trois lois de Newton comme fondement du moteur physique.
- Comprendre la démarche scientifique : modéliser, prédire, confronter à l'expérience.
- Préparer le lien entre forces et mouvement pour la simulation.

La démarche physique : modéliser l'univers

🔗 Physique du Jeu : Comment programmer l'univers

Isaac Newton n'a jamais codé en JavaScript, mais sans lui, aucun jeu vidéo moderne n'existerait. Les lois de Newton sont des modèles mathématiques basés sur l'observation. On les utilise tant qu'aucune expérience ne les invalide.

1. La Première Loi : L'Inertie

La Loi

Un objet au repos reste au repos, et un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit sur lui.

Ce que ça veut dire : Les objets résistent au changement. Pour modifier leur vitesse (accélérer, freiner, tourner), il faut appliquer une force.

Exemples :

- Bus qui freine : le corps continue d'avancer.
- **Space Invaders** : le personnage s'arrête instantanément quand on lâche la manette — physiquement faux, comme s'il y avait une friction infinie.
- **Asteroids** : le vaisseau continue en ligne droite sans friction — plus réaliste.

2. La Deuxième Loi : La Dynamique

💡 La Formule

$$\vec{F} = m \cdot \vec{a}$$

En pratique, on cherche l'accélération :

$$\vec{a} = \frac{\vec{F}}{m}$$

Ce que ça veut dire : L'accélération dépend de la force appliquée et de la masse de l'objet.

Exemples :

- Caddie de supermarché : vide, il part vite ; plein, il part lentement avec la même force.
- **Balancing FPS** : un Scout léger (50 kg) accélère à 10 m/s² avec 500 N, tandis qu'un Tank lourd (200 kg) n'accélère qu'à 2.5 m/s².

3. La Troisième Loi : Action-Réaction

La Loi

Pour toute action (force), il y a une réaction égale et opposée.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$$

Exemples :

- Skateboard : on pousse le sol vers l'arrière, le sol nous pousse vers l'avant.
- **Recul d'arme** : la balle part vers l'avant, l'arme recule vers l'arrière.
- **Rocket jump** : la roquette pousse le sol vers le bas, le joueur est propulsé vers le haut.
- **Collisions** : la boule blanche de billard transmet son impulsion à la boule rouge.

Résumé pour le développeur

Loi	Concept	Implémentation Code
1. Inertie	Les objets gardent leur vitesse.	Ne remettez pas velocity à 0 à chaque frame !
2. F = ma	Force -> Accélération.	<code>acc = forces / mass</code>
3. Action-Réaction	Les interactions sont doubles.	Collision : A repousse B, B repousse A.

Tableau 1. – Aide-mémoire des lois de Newton

Annexe : Pourquoi des objets de masses différentes tombent-ils avec la même vitesse ?

La force de pesanteur est le poids : $\vec{P} = m \cdot \vec{g}$.

La deuxième loi donne : $\vec{a} = \frac{\vec{F}}{m} = \frac{m \cdot \vec{g}}{m} = \vec{g}$.

La masse s'annule. L'accélération gravitationnelle est donc la même pour tous les objets.

Session 4 : Les forces de la nature

Objectifs de la session

- Identifier les forces de la nature agissant sur un objet dans un jeu.
- Comprendre le poids, la normale, le frottement sec et le frottement visqueux.
- Savoir quand et comment modéliser chaque force pour un gameplay réaliste.

Introduction

Dans le vide spatial, un objet lancé garde sa vitesse pour l'éternité (1ère loi de Newton). Sur Terre, tout finit par s'arrêter. Pourquoi ? À cause des interactions avec l'environnement, modélisées par les **forces de la nature**.

1. La Gravité et le Poids

La Gravité

La force d'attraction gravitationnelle attire les objets vers le centre de la Terre. Son accélération est notée $\vec{g}$.

$$\vec{P} = m \cdot \vec{g}$$

Application en jeu : La gravité est une force constante appliquée chaque frame à tous les objets dynamiques.

2. La Force Normale

La Force Normale (N)

C'est la force avec laquelle une surface repousse un objet qui appuie contre elle. Elle est perpendiculaire à la surface.

- Sur un sol plat horizontal : $N = mg$.
- Sur une pente : $N = mg \cos(\theta)$.

Application en jeu : La normale détermine si un objet est au sol et calcule la friction.

3. Le Frottement Sec (Modèle de Coulomb)

Frottement Sec

Frottement de contact entre deux solides. Il dépend du coefficient de friction μ et de la normale N .

$$\|\vec{f}\| = \mu \cdot \|\vec{N}\|$$

Statique vs Cinétique

- **Frottement statique (μ_s) :** l'objet est immobile. La force s'adapte jusqu'à un seuil critique $F_{\max} = \mu_s N$.

- **Frottement cinétique (μ_k)** : l'objet glisse. La force est constante $F_k = \mu_k N$.
- En général : $\mu_s > \mu_k$.

4. Le Frottement Visqueux (Fluide)

Frottement Visqueux

Résistance de l'air ou de l'eau. Il dépend de la vitesse et de la forme de l'objet.

- Vitesse faible : $\vec{f} = -k \cdot \vec{v}$.
- Vitesse élevée : $\vec{f} = -C \cdot \|v\|^2 \cdot \hat{v}$.

Conséquence : la vitesse terminale. Quand la gravité et le drag s'annulent, l'objet ne peut plus accélérer.

Exemple. Comparaison pour le Gameplay :

- **Frottement sec** : l'objet ralentit linéairement et s'arrête net.
- **Frottement fluide** : l'objet ralentit de moins en moins vite, approche asymptotique de 0.

Session 5 : Intégration numérique 101 - Méthode d'Euler

Objectifs de la session

- Comprendre pourquoi l'ordinateur discrétise le temps (frames, pas Δt).
- Apprendre la méthode d'Euler pour intégrer position et vitesse.
- Introduire la série de Taylor comme fondement des méthodes d'intégration.
- Identifier les limites d'Euler et quand l'utiliser malgré tout.

💡 Comment l'ordinateur prédit le futur

Dans nos simulations, le temps n'est pas continu. L'ordinateur calcule le monde image par image (souvent 60 FPS, soit $\Delta t \approx 0.016$ s). Les équations physiques continues (dérivées) doivent devenir des instructions discrètes (additions).

1. La Méthode d'Euler

L'Algorithme

Pour chaque pas de temps Δt :

1. Calculer l'accélération : $\vec{a}_n = \frac{\vec{F}}{m}$
2. Mettre à jour la vitesse : $\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$
3. Mettre à jour la position : $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$

Intuition : On suppose que la vitesse est constante pendant tout le pas de temps.

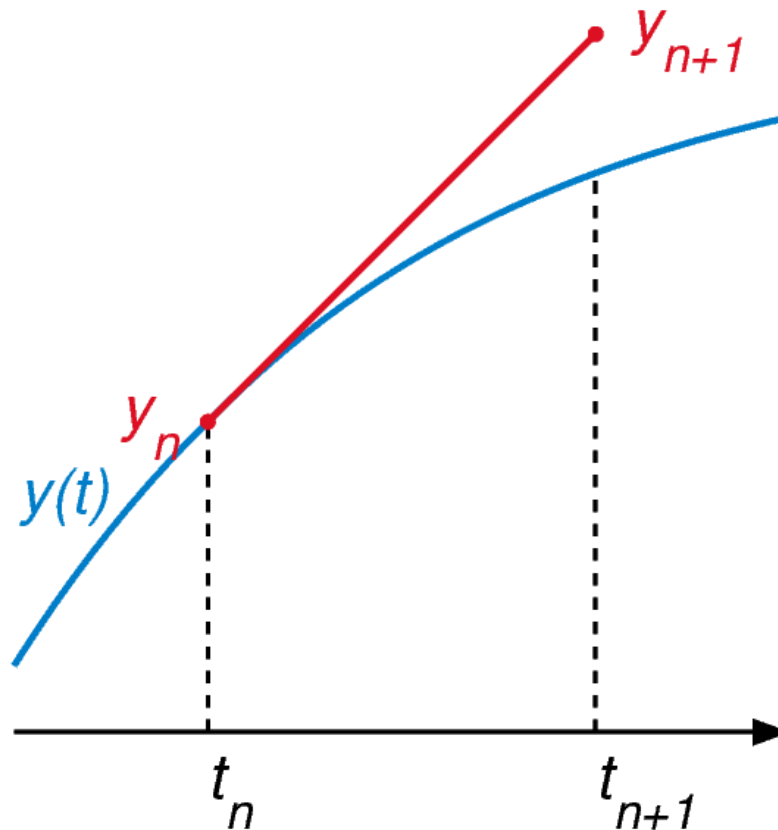


Fig. 8. – La méthode d'Euler : on avance pas à pas en supposant la vitesse constante pendant chaque Δt .

2. D'où ça vient ? La dérivée discrète

La vitesse est la dérivée de la position :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor

Approximation d'une fonction

La série de Taylor dit qu'une trajectoire peut être reconstruite en additionnant ses dérivées successives :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + \dots$$

Euler ne garde que le premier terme. C'est une méthode du **premier ordre**.

Pourquoi Euler est une approximation ?

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \underbrace{O(\Delta t^2)}_{\text{Erreur}}$$

4. Conséquences pratiques

- Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur du pas.
- Il tend à « dériver » vers l'extérieur des virages ou à gagner de l'énergie (voir Fig. 8).
- Malgré cela, il reste simple et rapide, et fonctionne quand l'accélération change (vent, ressorts, collisions).

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \frac{1}{2} \vec{a} \Delta t^2$ <i>Prend en compte la courbure.</i>
Facile à coder. Rapide.	Exact pour la gravité constante.

Tableau 2. – Comparaison pour un pas de temps Δt

5. Le code : euler_101.js

Nous allons maintenant décortiquer le fichier `examples/euler_101.js` : une simulation 3D d'une balle qui tombe et rebondit, propulsée par la méthode d'Euler. Le code est divisé en quatre couches : **l'état physique**, **l'initialisation Three.js**, **le cœur du moteur physique**, et **la boucle d'animation**.

5.1. L'état physique (variables globales)

Pourquoi des vecteurs mutables ?

Three.js fournit la classe `Vector3`. Au lieu d'en recréer un à chaque frame (`new THREE.Vector3(...)`), on en crée **un seul** qu'on modifie en place avec `.copy()`, `.add()`, `.set()`. C'est beaucoup plus doux pour le **garbage collector** du navigateur — un réflexe essentiel en boucle de jeu.

Paramètres et état initial.

```
// --- Paramètres Physique ---
const mass = 1.0;
const radius = 0.5;
const gravity = new THREE.Vector3(0, -9.81, 0);

// État initial de la balle (Position, Vitesse)
const startPosition = new THREE.Vector3(-5, 5, 0); // En haut à gauche
const startVelocity = new THREE.Vector3(3, 0, 0); // Lance vers la droite
const position = startPosition.clone();
const velocity = startVelocity.clone();
const force = new THREE.Vector3();
const acceleration = new THREE.Vector3();
```

- `mass`, `radius` : constantes de l'objet. La masse apparaît dans la 2^e loi de Newton $\vec{a} = \frac{\vec{F}}{m}$.

- gravity : le vecteur d'accélération gravitationnelle $\vec{g} = (0, -9.81, 0)$ « m/s »².
- startPosition / startVelocity : l'état à $t=0$, conservé pour pouvoir réinitialiser la simulation.
- position, velocity, force, acceleration : les **quatre vecteurs vivants** qui évoluent à chaque frame. On les clone pour ne pas écraser les valeurs de départ.

🔗 Les quatre vecteurs fondamentaux

Toute simulation newtonienne repose sur ces quatre quantités, mises à jour dans cet ordre exact à chaque pas de temps :

$$\vec{F} \rightarrow \vec{a} \rightarrow \vec{v} \rightarrow \vec{r}$$

Force $\vec{F}$, accélération $\vec{a}$, vitesse $\vec{v}$, position $\vec{r}$. C'est la **chaîne d'Euler**.

5.2. Initialisation Three.js (init)

La fonction `init()` construit la scène une seule fois au démarrage. Elle ne contient **aucune** physique — uniquement de la plomberie graphique.

Scène, caméra, rendu, lumières et sol.

```
function init() {
  scene = new THREE.Scene();
  scene.background = new THREE.Color(0xbfd1e5);
  camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 100);
  camera.position.set(0, 5, 15);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.shadowMap.enabled = true;
  document.body.appendChild(renderer.domElement);

  scene.add(new THREE.HemisphereLight(0xffeeee, 0xaaccaa));
  const light = new THREE.DirectionalLight(0xFFFF22, 1);
  light.position.set(5, 10, 7);
  light.castShadow = true;
  scene.add(light);

  const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(20, 20),
    new THREE.MeshStandardMaterial({ color: 0xaaaaaa })
  );
  floor.rotation.x = -Math.PI / 2;
  floor.receiveShadow = true;
  scene.add(floor);
  scene.add(new THREE.GridHelper(20, 20));

  createBall();
  setupGUI();
  new OrbitControls(camera, renderer.domElement);
  window.addEventListener('resize', onWindowResize, false);
  renderer.setAnimationLoop(animate);
}
```


- **Scène & caméra** : la caméra PerspectiveCamera avec un champ de vision de 50°, placée en (0, 5, 15) pour voir la scène de biais.
- **Render** : WebGLRenderer dessine sur la page. antialias: true lisse les arêtes. shadowMap.enabled active les ombres portées.
- **Lumières** : une HemisphereLight (ciel + sol) pour l’ambiance, et une DirectionalLight (le « soleil ») qui projette des ombres.
- **Sol** : un PlaneGeometry couché par rotation.x = -Math.PI / 2 (rotation de -90° autour de l’axe X pour passer du plan XY au plan XZ). receiveShadow lui permet de recevoir l’ombre de la balle.
- **Lancement** : renderer.setAnimationLoop(animate) demande au navigateur d’appeler animate à chaque frame (≈ 60 FPS).

5.3. La balle et ses flèches (createBall)

Maillage + flèches de débogage.

```
function createBall() {
  ball = new THREE.Mesh(
    new THREE.SphereGeometry(radius, 32, 32),
    new THREE.MeshStandardMaterial({ color: 0x0077ff })
  );
  ball.castShadow = true;
  scene.add(ball);

  velArrow = new THREE.ArrowHelper(new THREE.Vector3(1, 0, 0), position, 1, 0x333333);
  scene.add(velArrow);

  accArrow = new THREE.ArrowHelper(new THREE.Vector3(0, 1, 0), position, 1, 0xAA8800);
  scene.add(accArrow);
}
```

- ball : un Mesh = une SphereGeometry (le rayon vient de notre variable physique) + un matériau bleu. castShadow projette son ombre sur le sol.
- velArrow / accArrow : deux ArrowHelper qui affichent en temps réel la **direction** et la **magnitude** de la vitesse (noir) et de l’accélération (jaune). Indispensables pour **voir** la physique — sans eux, Euler est une boîte noire.

5.4. Le cœur physique : updatePhysics(dt)

C’est ici que la méthode d’Euler prend vie. La fonction suit **exactement** les trois étapes de l’algorithme vu en section 1, plus une quatrième pour gérer le rebond.

Étape 1 — Calcul des forces.

```
function updatePhysics(dt) {
  // 1. Calcul des Forces
  force.set(0, 0, 0); // Reset

  // Ajout de la gravité (P = m * g)
  const gravityForce = gravity.clone().multiplyScalar(mass);
  force.add(gravityForce);
}
```

On **remet à zéro** la force à chaque frame (force.set(0,0,0)) — sinon les forces s’accumuleraient d’une frame à l’autre et la balle s’envolerait. Puis on ajoute le poids $\vec{P} = m \cdot \vec{g}$. Ici, seule la gravité agit ; dans un vrai jeu, on empilerait ici ressorts, vent, frottements, etc.

Étape 2 — Deuxième loi de Newton.

```
// 2. Deuxième Loi de Newton : a = F / m
acceleration.copy(force).divideScalar(mass);
```

On calcule $\vec{a} = \frac{\vec{F}}{m}$. `.copy(force)` recopie la force dans `acceleration`, puis `.divideScalar(mass)` divise par la masse. Avec $m = 1$, on a $\vec{a} = \vec{g}$, ce qui est cohérent : en chute libre, tout tombe à la même accélération.

Étape 3 — Intégration d'Euler.

```
// 3. Intégration d'Euler (Premier ordre)
// v_nouveau = v_actuel + a * dt
velocity.addScaledVector(acceleration, dt);

// p_nouveau = p_actuel + v * dt
position.addScaledVector(velocity, dt);
```

Ce sont **exactement** les deux formules de la définition de la section 1 :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$$

 Subtilité : la vitesse utilisée pour la position

Remarquez qu'on met à jour la vitesse **avant** la position, et qu'on utilise la **nouvelle** vitesse $\vec{v}_{n+1}$ pour intégrer la position. C'est la variante **semi-implicite** (ou **symplectique**) d'Euler. Elle est légèrement plus stable que l'Euler classique (qui utiliserait $\vec{v}_n$) et conserve mieux l'énergie sur le long terme. C'est le choix par défaut dans la plupart des moteurs de jeu.

Étape 4 — Collisions avec le sol.

```
// 4. Gestion des Collisions (Sol)
if (position.y < radius) {
    position.y = radius;
    velocity.y = -velocity.y * params.restitution;
    velocity.x *= 0.95;
    velocity.z *= 0.95;
}
```

- **Test** : si le centre de la balle descend sous l'altitude `radius`, le bas de la sphère touche le sol.
- **Correction de position** : on remonte la balle à `position.y = radius` pour éviter qu'elle ne s'enfonce ou tremble sous le plan.
- **Rebond** : on inverse la composante verticale de la vitesse et on la multiplie par le coefficient de restitution e (`params.restitution`). Avec $e = 1$ la balle rebondit à la même hauteur ; avec $e = 0$ elle s'écrase ; à $e = 0.8$ elle perd 20 % d'énergie à chaque rebond.
- **Frottement tangentiel** : on amortit légèrement les composantes horizontales ($*0.95$) pour simuler le frottement au contact — sinon la balle glisserait indéfiniment.

💡 Pourquoi un test simple suffit ici

On teste uniquement $\text{position.y} < \text{radius}$ parce que le sol est un plan infini à $y = 0$. Pour des collisions entre **deux sphères**, le test deviendrait $\|\vec{r}_A - \vec{r}_B\| < r_A + r_B$ — c'est-à-dire que la distance entre les centres est inférieure à la somme des rayons. On verra ça en Session 6.

5.5. Synchronisation visuelle (updateVisuals)

La physique calcule des nombres ; Three.js affiche des objets. `updateVisuals` fait le pont.

Copier la physique vers le visuel.

```
function updateVisuals() {
  ball.position.copy(position);

  velArrow.position.copy(position);
  velArrow.setDirection(velocity.clone().normalize());
  velArrow.setLength(velocity.length() * 0.3, 0.2, 0.1);

  accArrow.position.copy(position);
  if (acceleration.lengthSq() > 0.0001) {
    accArrow.setDirection(acceleration.clone().normalize());
    accArrow.setLength(acceleration.length() * 0.3, 0.2, 0.1);
  }
}
```

- `ball.position.copy(position)` : on **copie** le vecteur physique dans le mesh — jamais d'affectation directe (`ball.position = position` casserait la référence interne de Three.js).
- **Flèche vitesse** : `.normalize()` donne la direction (vecteur unitaire), `.length()` donne la magnitude. La longueur de la flèche est proportionnelle à $\|\vec{v}\|$, donc la flèche **grandit** quand la balle accélère.
- **Flèche accélération** : on vérifie `lengthSq() > 0.0001` avant de normaliser pour éviter une division par zéro (un vecteur nul n'a pas de direction). Ici l'accélération est constante (gravité), donc la flèche pointe toujours vers le bas avec la même longueur.

5.6. La boucle d'animation (animate)

Le pouls de la simulation.

```
function animate() {
  const dt = clock.getDelta();
  const safeDt = Math.min(dt, 0.1) * params.timeScale;
  updatePhysics(safeDt);
  updateVisuals();
  renderer.render(scene, camera);
}
```

💡 Le `dt` est le cœur d'Euler

`clock.getDelta()` renvoie le temps écoulé **en secondes** depuis la dernière frame — c'est notre Δt . Sans lui, la simulation tournerait à des vitesses différentes selon la machine (144 Hz vs 60 Hz vs 30 Hz). Avec `dt`, la physique est **indépendante du framerate** : la balle met le même temps à tomber sur un vieux portable que sur un écran de gamer.

- **Clamp** : `Math.min(dt, 0.1)` plafonne Δt à 0,1 s. Si l'utilisateur change d'onglet, le navigateur suspend la boucle ; au retour, `getDelta()` renverrait un Δt énorme (plusieurs secondes) qui ferait **exploser** la simulation (la balle traverserait le sol). Le clamp évite ce piège classique.
- **timeScale** : `params.timeScale` permet de ralentir (0,5×), accélérer (2×) ou mettre en pause (0×) la simulation sans toucher au code physique — utile pour observer un phénomène.
- **Ordre** : `updatePhysics` → `updateVisuals` → `render`. On calcule l'état, on le reflète sur les objets, puis on dessine. Cet ordre garantit que l'image affichée correspond **toujours** à l'état physique le plus récent.

6. Résumé : la chaîne complète

De la force à l'image, en une frame

À chaque image, `animate` déclenche cette cascade :

1. `clock.getDelta()` → mesure Δt réel.
2. `updatePhysics(dt)` :
 - Calcule $\vec{F}$ (gravité).
 - $\vec{a} = \frac{\vec{F}}{m}$.
 - $\vec{v}_{n+1} = \vec{v}_n + \vec{a} \cdot \Delta t$ (Euler).
 - $\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \cdot \Delta t$.
 - Teste et résout la collision avec le sol.
3. `updateVisuals()` → copie $\vec{r}$ et $\vec{v}$ vers le mesh et les flèches.
4. `render.render()` → dessine la frame à l'écran.

C'est exactement le schéma de la Fig. 8, traduit en JavaScript. Tout l'art d'un moteur de jeu tient dans cette boucle, exécutée 60 fois par seconde.

Session 6 : L'impulsion, les collisions

Objectifs de la session

- Comprendre l'impulsion comme changement de quantité de mouvement.
- Apprendre la loi de restitution et la vitesse relative.
- Calculer l'impulsion d'une collision entre deux objets.
- Implémenter un laboratoire de chocs 1D.

1. La Quantité de Mouvement

Quantité de mouvement (p)

La quantité de mouvement est le produit de la masse par la vitesse d'un objet :

$$\vec{p} = m \cdot \vec{v}$$

- C'est une grandeur **vectorielle** : elle a la même direction que la vitesse.
- Unité SI : $\text{kg} \cdot \text{m/s}$.
- Elle mesure « combien de mouvement » porte un objet — un camion lent et une balle rapide peuvent avoir la même quantité de mouvement.

💡 Conservation de la quantité de mouvement

Dans un système isolé (sans forces externes), la quantité de mouvement **totale** est conservée :

$$\vec{p}_{\text{totale}} = \vec{p}_A + \vec{p}_B = \text{constante}$$

C'est la loi fondamentale qui gouverne les collisions : ce qui est perdu par un objet est gagné par l'autre.

2. D'où vient l'Impulsion ?

Théorème de l'Impulsion

La force est liée à la variation de la vitesse (si la masse est constante) :

$$\vec{F} = m \cdot \frac{d\vec{v}}{dt}$$

Comme $\vec{p} = m \cdot \vec{v}$, cela s'écrit aussi :

$$\vec{F} = \frac{d\vec{p}}{dt}$$

En intégrant sur un choc instantané ($\Delta t \approx 0$) :

$$\vec{J} = \Delta\vec{p} = m \cdot \Delta\vec{v}$$

Comment un choc bref change-t-il la vitesse si vite ?

D'après $\vec{F} = m \cdot \frac{d\vec{v}}{dt}$, on isole la variation de vitesse :

$$\Delta \vec{v} = \frac{\vec{F} \cdot \Delta t}{m}$$

Si Δt est très petit (un choc quasi instantané), il faut une force **énorme** pour produire un $\Delta \vec{v}$ significatif. Mais le produit $\vec{F} \cdot \Delta t$ reste fini — c'est l'impulsion $\vec{J}$.

- **Exemple :** Une balle de tennis frappée par une raquette. Le contact dure ≈ 5 ms, mais la force atteint ≈ 1000 N. L'impulsion vaut $\vec{J} = 1000 \cdot 0.005 = 5 \text{ kg} \cdot \text{m/s}$ — assez pour faire passer la balle de 0 à 50 m/s.
- L'accélération est bel et bien gigantesque ($a = \frac{F}{m} \approx 20000 \text{ m/s}^2$), mais elle ne dure qu'un instant. Ce qui compte, c'est le **produit** $F \cdot \Delta t$, pas la force seule.

💡 Intuition clé

Ce n'est pas la force seule qui change la vitesse, ni le temps seul. C'est leur **produit** — l'impulsion. Une petite force appliquée longtemps peut produire le même changement de vitesse qu'une force énorme appliquée très brièvement.

💡 Pourquoi on n'utilise pas Euler pour les collisions

La méthode d'Euler (Session 5) calcule $\Delta \vec{v} = \vec{a} \cdot \Delta t$ à chaque pas de temps. Mais lors d'un choc, la force (et donc l'accélération) est énorme et dure un temps négligeable. Avec $\Delta t \approx 0.016$ s, soit on rate complètement le choc (l'objet traverse), soit on surestime grossièrement la force. Au lieu de simuler la force frame par frame, on **saute directement au résultat** : on applique l'impulsion $\vec{J}$ qui change la vitesse instantanément.

3. La Loi de Restitution

Coefficient de restitution (e)

La vitesse à laquelle deux objets s'éloignent après un choc est proportionnelle à la vitesse à laquelle ils se rapprochaient avant le choc.

$$v_{\text{rel}}(\text{après}) = -e \cdot v_{\text{rel}}(\text{avant})$$

- $e = 1$: choc élastique (billes de billard).
- $e = 0$: choc mou (pâte à modeler).

4. Vitesse Relative et Normale

💡 Point de départ : le test de collision

Tout choc commence par un test de collision : détecter que deux objets se chevauchent. Une fois le contact trouvé, seule la vitesse **projetée sur la normale de collision** $\vec{n}$ compte. C'est elle qui détermine si les objets se rapprochent, s'éloignent ou glissent l'un contre l'autre.

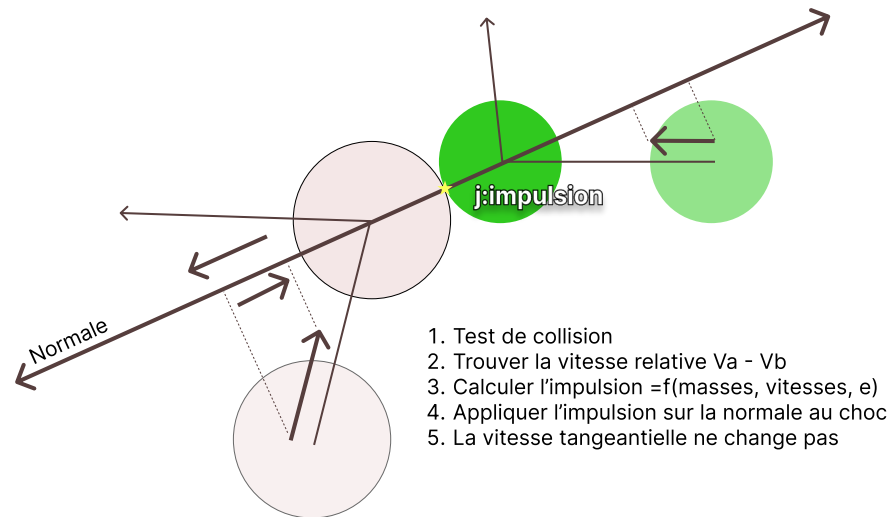


Fig. 9. – Vue schématique d'une collision : normale de contact, vitesses avant et impulsion appliquée.

Vitesse relative scalaire

La vitesse de rapprochement projetée sur la normale de collision :

$$v_{\text{rel}} = (\vec{v}_A - \vec{v}_B) \cdot \vec{n}$$

- $v_{\text{rel}} < 0$: les objets se rapprochent.
- $v_{\text{rel}} > 0$: les objets s'éloignent.
- $v_{\text{rel}} = 0$: ils glissent l'un contre l'autre.

5. Calcul de l'Impulsion

🗨 Formule de l'impulsion

$$j = \underbrace{-(1 + e)v_{\text{rel}}}_{\text{Vitesse à changer}} \cdot \underbrace{\left(\frac{m_A m_B}{m_A + m_B} \right)}_{\text{Masse réduite}}$$

Application aux vitesses :

$$v'_A = v_A + \left(\frac{j}{m_A} \right) \cdot \vec{n}$$

$$v'_B = v_B - \left(\frac{j}{m_B} \right) \cdot \vec{n}$$

💡 Vitesse tangentielle inchangée

L'impulsion $\vec{J}$ est appliquée le long de la normale $\vec{n}$. Seule la composante **normale** de la vitesse est modifiée. La composante **tangentielle** (perpendiculaire à $\vec{n}$) reste inchangée : $\vec{v}_{\text{tangentielle}}' =$

$\vec{v}_{\text{tangentielle}}$

6. TP : Laboratoire de Chocs 1D

🔗 Objectifs du TP

Implémenter la réponse physique d'une collision entre deux boules de masses différentes dans un espace 1D sans gravité ni friction.

Missions

1. Créer deux boules : A ($m = 5$) lancée vers B ($m = 1$).
2. Calculer la normale $\vec{n}$.
3. Calculer la vitesse relative v_{rel} .
4. Si $v_{\text{rel}} > 0$, sortir (les objets s'éloignent).
5. Calculer l'impulsion j avec la masse réduite.
6. Appliquer le changement de vitesse aux deux boules.

Scénarios à observer

- **Pendule de Newton** : masses égales, $e = 1$ — A s'arrête, B repart.
- **Le Mur** : $m_A = 1$, $m_B = 100$ — A rebondit, B bouge à peine.
- **Bulldozer** : $m_A = 100$, $m_B = 1$ — A continue, B repart à environ $2v_A$.
- **Pâte à modeler** : $e = 0$ — les boules se collent.

Session 7 : Broad et Narrow phases – Part 1

Objectifs de la session

- Comprendre le **pipeline de détection de collision** en trois étapes.
- Mesurer le budget temps d'une frame (16 ms) et pourquoi la physique doit tenir en 2 ms.
- Expliquer pourquoi la détection naïve est en $O(N^2)$ et pourquoi c'est inacceptable.
- Maîtriser l'AABB (Axis Aligned Bounding Box) comme fondement de la Broad Phase.
- Détailler trois structures de partitionnement : Grille Uniforme, Quadtree/Octree, Sweep-and-Prune.
- Comparer leurs complexités, avantages et cas d'usage.
- Introduire la Narrow Phase et le test de chevauchement AABB.

1. Le Pipeline de Détection de Collision

Trois étapes pour détecter et résoudre une collision

Un moteur physique moderne ne teste pas toutes les paires d'objets. Il suit un **pipeline en entonnoir** qui élimine progressivement les faux positifs :

1. **Broad Phase** — « Qui pourrait entrer en collision ? »
Réponse rapide avec des boîtes simplifiées (AABB). Élimine 95–99 % des paires.
2. **Narrow Phase** — « Ces deux objets se touchent-ils vraiment ? »
Test géométrique précis (sphère, OBB, GJK, etc.). Coûteux mais appliqué à très peu de paires.
3. **Collision Response** — « Que faire ? »
Impulsion, correction de position, friction (vu en Session 6).

💡 L'entonnoir en chiffres

Pour 1 000 objets, la brute force génère $\approx 500\,000$ paires. La Broad Phase en retient typiquement ≈ 2000 . La Narrow Phase confirme ≈ 200 collisions réelles. La Response n'en traite que les pertinentes. Chaque étage divise le travail par un ordre de grandeur.

2. Le Contexte : Le Budget Temps (16 ms)

Le budget temps d'une frame

À 60 FPS, on dispose de **16.6 millisecondes** pour tout calculer :

- Rendu : 8 ms
- IA / Gameplay : 4 ms
- Audio / Réseau : 2 ms
- **Physique : 2 ms**

Si la détection de collision prend 10 ms, le jeu saccade. L'optimisation est vitale.

! Les 2 ms de la physique

Sur ces 2 ms, la Broad Phase doit prendre **moins de 0.5 ms**. C'est elle qui scale avec N . La Narrow Phase et la Response consomment le reste. Si la Broad Phase est lente, tout le reste cascade — c'est le goulot d'étranglement numéro un des moteurs physiques.

3. Le Problème : La Malédiction $O(N^2)$

Détection naïve (brute force)

Avec N objets, chaque objet teste tous les autres. Le nombre de paires uniques est :

$$C = \frac{N(N-1)}{2} \approx O(N^2)$$

Objets (N)	Paires	Temps à 1 $\mu\text{s}/\text{test}$
10	45	$\approx 0.05 \text{ ms}$
100	4 950	$\approx 5 \text{ ms}$
1 000	499 500	$\approx 500 \text{ ms}$
10 000	49 995 000	$\approx 50 \text{ s}$

Tableau 3. – La croissance quadratique rend la brute force inutilisable au-delà de quelques dizaines d'objets.

Solution : réduire le nombre de paires **avant** les tests coûteux. C'est le rôle de la **Broad Phase**.

4. L'AABB : La Boîte Fondamentale

Axis Aligned Bounding Box (AABB)

Une AABB est le plus petit rectangle (2D) ou pavé (3D) **aligné sur les axes** qui contient un objet. Elle est définie par deux points :

$$\text{AABB} = \{\min(x_{\min}, y_{\min}, z_{\min}), \max(x_{\max}, y_{\max}, z_{\max})\}$$

💡 Pourquoi des boîtes alignées sur les axes ?

Parce que le test de chevauchement entre deux AABB est **trivial** — quelques comparaisons sur chaque axe, pas de produit vectoriel, pas de trigonométrie. C'est le test le plus rapide qui existe en 3D.

Test de chevauchement AABB vs AABB

Deux AABB se chevauchent si et seulement si leurs intervalles se chevauchent sur **tous** les axes :

$$\text{overlap}_x = (x_{\max}^A \geq x_{\min}^B) \wedge (x_{\max}^B \geq x_{\min}^A)$$

Si un seul axe ne se chevauche pas, il n'y a pas de collision. C'est le principe du **Separating Axis**.

Test AABB en JavaScript.

```
function aabbOverlap(a, b) {
  // A et B sont des objets { min: Vector3, max: Vector3 }
  if (a.max.x < b.min.x || b.max.x < a.min.x) return false;
  if (a.max.y < b.min.y || b.max.y < a.min.y) return false;
  if (a.max.z < b.min.z || b.max.z < a.min.z) return false;
  return true; // Chevauchement sur les 3 axes
}
```

Calcul d'une AABB pour une sphère : trivial — la boîte va de $c - r$ à $c + r$ sur chaque axe, où c est le centre et r le rayon.

Calcul d'une AABB pour un mesh complexe : on parcourt tous les sommets et on garde les min et max par axe. Ce calcul se fait **une fois** au chargement, puis on translate la boîte avec l'objet.

5. La Broad Phase : Structures de Partitionnement

Principe de la Broad Phase

On entoure chaque objet par son AABB, puis on utilise une **structure de partitionnement** pour trouver rapidement quelles paires d'AABB **pourraient** se chevaucher. On ne teste précisément que les candidates.

On présente ici les trois structures les plus utilisées en jeu vidéo.

5.A. Grille Uniforme (Spatial Hashing)

Principe

On découpe le monde en cases (cells) de taille fixe S . Chaque objet est rangé dans la case correspondant à sa position. Pour tester les collisions d'un objet, on ne regarde que les objets dans sa case et les cases **adjacentes** (8 en 2D, 26 en 3D).

Spatial Hashing — structure et requête.

```
class SpatialGrid {
  constructor(cellSize) {
    this.cellSize = cellSize;
    this.cells = new Map(); // clé "x,y,z" -> liste d'objets
  }

  getKey(pos) {
    const x = Math.floor(pos.x / this.cellSize);
    const y = Math.floor(pos.y / this.cellSize);
    const z = Math.floor(pos.z / this.cellSize);
    return `${x},${y},${z}`;
  }

  clear() { this.cells.clear(); }

  insert(obj) {
```

```

const key = this.getKey(obj.position);
if (!this.cells.has(key)) this.cells.set(key, []);
this.cells.get(key).push(obj);
}

getNeighbors(pos) {
  const cx = Math.floor(pos.x / this.cellSize);
  const cy = Math.floor(pos.y / this.cellSize);
  const cz = Math.floor(pos.z / this.cellSize);
  const neighbors = [];
  for (let dx = -1; dx <= 1; dx++)
    for (let dy = -1; dy <= 1; dy++)
      for (let dz = -1; dz <= 1; dz++) {
        const cell = this.cells.get(`${cx+dx},${cy+dy},${cz+dz}`);
        if (cell) neighbors.push(...cell);
      }
  return neighbors;
}
}

```

🔗 Choix de la taille de cellule

La taille idéale est $S \approx$ diamètre max des objets. Si S est trop petit, un objet chevauche plusieurs cases (overhead). Si S est trop grand, chaque case contient trop d'objets (le filtrage ne sert à rien).

- **Complexité** : $O(N)$ pour construire la grille, $O(K)$ par objet où K est le nombre d'objets dans les 27 cases voisines. Si la distribution est uniforme, K est constant $\rightarrow$ **total en $O(N)$** .
- **Avantage** : accès $O(1)$, très rapide, facile à coder, excellent pour des objets de taille similaire.
- **Inconvénient** : mauvais si les tailles varient beaucoup (un petit objet et un énorme bâtiment dans la même grille), ou si le monde est infini (il faut un dictionnaire au lieu d'un tableau).
- **Cas d'usage typique** : particules, billes, projectiles, jeux d'arène fermée.

5.B. Arbres Spatiaux (Quadtree / Octree)

Principe

On commence avec une grande boîte contenant tout le monde. Si elle contient plus de T objets (seuil), on la découpe en 4 (2D = Quadtree) ou 8 (3D = Octree) sous-boîtes égales. On recommence récursivement jusqu'à ce que chaque feuille contient au plus T objets.

Octree — insertion et requête.

```

class OctreeNode {
  constructor(min, max, depth = 0, maxDepth = 5, maxObjects = 8) {
    this.min = min; this.max = max;
    this.depth = depth;
    this.maxDepth = maxDepth;
    this.maxObjects = maxObjects;
    this.objects = [];
    this.children = null; // 8 sous-nœuds ou null
  }

  insert(obj) {

```

```

    if (this.children) {
        for (const child of this.children)
            if (child.contains(obj)) child.insert(obj);
        return;
    }
    this.objects.push(obj);
    if (this.objects.length > this.maxObjects
        && this.depth < this.maxDepth) {
        this.subdivide();
        // Redistribuer les objets existants
        for (const o of this.objects)
            for (const child of this.children)
                if (child.contains(o)) child.insert(o);
        this.objects = [];
    }
}

subdivide() {
    const mid = this.min.clone().add(this.max).multiplyScalar(0.5);
    // Créer 8 enfants (2^3 combinaisons de min/max par axe)
    this.children = [];
    for (let i = 0; i < 8; i++) {
        const childMin = new THREE.Vector3(
            (i & 1) ? mid.x : this.min.x,
            (i & 2) ? mid.y : this.min.y,
            (i & 4) ? mid.z : this.min.z
        );
        const childMax = new THREE.Vector3(
            (i & 1) ? this.max.x : mid.x,
            (i & 2) ? this.max.y : mid.y,
            (i & 4) ? this.max.z : mid.z
        );
        this.children.push(new OctreeNode(childMin, childMax, this.depth + 1,
            this.maxDepth, this.maxObjects));
    }
}

query(aabb, results = []) {
    if (!this.intersectsAABB(aabb)) return results;
    if (this.children) {
        for (const child of this.children) child.query(aabb, results);
    } else {
        results.push(...this.objects);
    }
    return results;
}
}

```

🗨 Reconstruction vs mise à jour

Dans un jeu où les objets bougent, deux stratégies existent :

- **Rebuild chaque frame** : on détruit et reconstruit l'arbre. Simple mais coûteux ($O(N \log N)$). Acceptable si N est modéré (< 10000).

- **Update incrémental** : on déplace les objets dans l'arbre. Plus complexe, mais nécessaire pour de très grands mondes.

La plupart des moteurs de jeu font un **rebuild** chaque frame : c'est plus simple et souvent plus rapide en pratique grâce au cache.

- **Complexité** : $O(N \log N)$ pour construire, $O(\log N + K)$ pour une requête.
- **Avantage** : s'adapte automatiquement à la densité — zones denses subdivisées davantage, zones vides non subdivisées.
- **Inconvénient** : coûteux à reconstruire/mettre à jour si les objets bougent beaucoup ; overhead mémoire pour les nœuds.
- **Cas d'usage typique** : mondes ouverts, objets de tailles variées, culling de frustum, raycasting.

5.C. Sweep and Prune (SAP)

Principe

On projette les débuts (min) et fins (max) de chaque AABB sur un axe (généralement X). On trie la liste des intervalles. En parcourant la liste triée, on maintient un ensemble « actif » d'intervalles qui n'ont pas encore été fermés. Deux intervalles actifs simultanément se chevauchent sur cet axe.

Sweep and Prune — algorithme 1D.

```
class SweepAndPrune {
  constructor() {
    this.intervals = []; // [{ id, value, isStart }]
  }

  update(objects) {
    this.intervals = [];
    for (const obj of objects) {
      this.intervals.push({ id: obj.id, value: obj.aabb.min.x, isStart: true });
      this.intervals.push({ id: obj.id, value: obj.aabb.max.x, isStart: false });
    }
    // Tri par position croissante
    this.intervals.sort((a, b) => a.value - b.value);
  }

  getPairs() {
    const pairs = [];
    const active = new Set();
    for (const interval of this.intervals) {
      if (interval.isStart) {
        // Cet objet démarre : il chevauche tous les actifs
        for (const id of active)
          pairs.push([id, interval.id]);
        active.add(interval.id);
      } else {
        active.delete(interval.id);
      }
    }
    return pairs;
  }
}
```

```

}
}

```

💡 Cohérence temporelle (temporal coherence)

D'une frame à l'autre, les objets bougent peu. La liste triée est donc **presque** la même. Au lieu d'un tri complet ($O(N \log N)$), on utilise un **tri par insertion** ($O(N)$ sur une liste presque triée). C'est ce qui rend SAP si rapide en pratique : le coût amorti est quasi linéaire.

💬 SAP multi-axe (SAP 3D)

La version 1D génère des **faux positifs** : deux objets peuvent se chevaucher sur X mais pas sur Y. La version 3D exécute SAP sur les trois axes et ne retient que les paires qui se chevauchent sur **tous** les axes. On peut aussi n'utiliser qu'un seul axe (le plus discriminant) et filtrer ensuite avec le test AABB complet.

- **Complexité** : $O(N \log N)$ au pire, $O(N)$ amorti grâce à la cohérence temporelle.
- **Avantage** : exploite la cohérence temporelle, pas de paramètre de taille de cellule à régler, excellent pour des objets de tailles variées.
- **Inconvénient** : dégénère si tous les objets sont alignés sur le même axe (la liste active devient énorme).
- **Cas d'usage typique** : moteurs physiques professionnels (Bullet, Box2D utilise une variante), objets de tailles hétérogènes.

6. Tableau Comparatif des Structures

Critère	Grille Uniforme	Quadtree/Octree	SAP
Complexité construction	$O(N)$	$O(N \log N)$	$O(N \log N) / O(N)$ amorti
Complexité requête	$O(K)$ par objet	$O(\log N + K)$	$O(N + P)$ paires
Tailles hétérogènes	Mauvais	Bon	Bon
Monde infini	Possible (hash)	Difficile	Oui
Objets statiques	Excellent	Excellent	Bon
Objets dynamiques	Très bon	Coûteux (rebuild)	Excellent (cohérence)
Paramètres à régler	Taille de cellule	Seuil, profondeur max	Aucun
Difficulté d'implémentation	Facile	Moyenne	Moyenne

Tableau 4. – Comparaison des trois structures de Broad Phase les plus courantes. K = objets voisins, P = paires candidates.

7. Introduction à la Narrow Phase

Le rôle de la Narrow Phase

La Broad Phase donne une liste de **paires candidates**. La Narrow Phase teste chaque paire avec une géométrie **précise** pour confirmer la collision et calculer la normale de contact.

Tests courants (du moins cher au plus cher) :

- **Sphère vs Sphère** : $\|c_A - c_B\| < r_A + r_B$. Un seul test, $O(1)$.
- **AABB vs AABB** : 6 comparaisons (vu section 4).
- **Sphère vs AABB** : distance du centre au boîtier, clampé sur chaque axe.
- **OBB vs OBB** (Oriented Bounding Box) : théorème des axes séparateurs (SAT), jusqu'à 15 axes en 3D.
- **Maillage vs Maillage** : GJK (Gilbert–Johnson–Keerthi) + EPA pour la pénétration. Coûteux, réservé aux paires confirmées.

💡 Hiérarchie de tests

On applique toujours les tests du **moins cher au plus cher**. Si le test AABB échoue, on s'arrête — inutile de lancer GJK. C'est la même philosophie que l'entonnoir : chaque étage élimine des candidats à moindre coût.

8. TP : Broad Phase — Spatial Hashing

💡 Objectif du TP

Implémenter une Broad Phase par **grille uniforme (Spatial Hashing)** et le test de chevauchement **AABB**, puis comparer les performances avec la brute force $O(N^2)$. Le fichier de départ `session07_Broad_Part_1.js` fournit la scène Three.js, la GUI et le moteur de collision — il ne reste qu'à compléter trois fonctions.

Mise en place

- Ouvrir `session07_Broad_Part_1.html` dans le navigateur (via Live Server).
- La scène contient un cube 3D rempli de billes en mouvement avec rebonds sur les parois.
- La GUI propose : nombre de billes (10–3000), toggle Broad Phase ON/OFF, restitution, vitesse du temps, reset.
- Un compteur **FPS** et **Tests/frame** est affiché en temps réel.
- Au départ, la Broad Phase est activée mais les fonctions sont vides — tout passe en brute force. C'est normal, c'est ce que vous allez corriger.

Missions

Mission 1 — `SpatialGrid.insert(obj)`

Ranger chaque objet dans la bonne cellule de la grille. Utiliser `getKey(obj.position)` pour obtenir la clé "x,y,z", créer le tableau si la cellule n'existe pas, puis ajouter l'objet.

Mission 2 — `SpatialGrid.getNeighbors(pos)`

Récupérer les objets situés dans la cellule de `pos` et les **26 cellules voisines** ($3 \times 3 \times 3$). Boucler sur `dx`, `dy`, `dz` de -1 à +1, construire la clé et récupérer le contenu. Retourner le tableau des voisins.

Mission 3 — `aabbOverlap(aabbA, aabbB)`

Implémenter le test de chevauchement AABB vu en section 4 : 6 comparaisons, une par axe. Renvoyer `false` dès qu'un axe ne se chevauche pas, `true` si les trois se chevauchent.

Mission 4 — Benchmark

Une fois les trois fonctions complétées, comparer Broad Phase ON vs OFF avec différents nombres de billes. Observer le compteur **Tests/frame** : il doit chuter drastiquement quand la Broad Phase est active.

Scénarios à observer

- **Faible densité (50 billes)** : la brute force reste fluide. La Broad Phase est légèrement plus rapide mais la différence est faible.
- **Forte densité (1000+ billes)** : la brute force s'effondre (≈ 500000 tests/frame), la Broad Phase reste fluide.
- **Extrême (3000 billes)** : avec Broad Phase ON, la simulation reste interactive ; avec OFF, elle devient un diaporama. C'est la démonstration concrète de l'entonnoir.
- **Compteur Tests/frame** : avec Broad Phase, le nombre de tests doit être proportionnel au nombre de **voisins réels**, pas à N^2 . Comparer les chiffres affichés.

⚠ Pièges à éviter

- Ne pas oublier d'appeler `grid.clear()` au début de chaque frame avant de réinsérer les objets.
- La fonction `getNeighbors` doit couvrir les **27** cellules ($3 \times 3 \times 3$), pas seulement la cellule courante — un objet à cheval entre deux cellules doit être détecté.
- Le test AABB doit renvoyer `false` **dès le premier axe qui ne se chevauche pas** — ne pas tester les trois axes si le premier échoue (optimisation).
- Faire attention aux **doublons** : le code fourni utilise `ballB.id <= ballA.id` pour éviter de tester chaque paire deux fois. Ne pas retirer cette garde.

Bonus

Ajouter un **Octree** comme troisième option dans la GUI (en plus de Brute Force et Grille). Comparer les performances lorsque les billes sont concentrées dans un coin du cube (distribution non uniforme). L'Octree devrait mieux gérer cette situation car il subdivise les zones denses.

Session 8 : Intégration de Verlet et Position Based Dynamics

Objectifs de la session

- Comprendre **pourquoi** l'intégration d'Euler explose sur les systèmes de particules (ressorts, tissus).
- Maîtriser l'intégration de **Verlet** : vitesse implicite, stabilité, dérivation de la formule.
- Appliquer un **damping** adapté à Verlet pour dissiper l'énergie.
- Découvrir **Position Based Dynamics** (PBD) : résoudre des contraintes de distance en manipulant directement les positions.
- Assembler ces briques dans un TP : un drapeau animé en tissu.

1. Le Problème : Pourquoi Euler Explode

Rappel : intégration d'Euler explicite

À chaque pas de temps, Euler met à jour position et vitesse séparément :

$$x_{n+1} = x_n + v_n \cdot dt \quad ; \quad v_{n+1} = v_n + a_n \cdot dt$$

C'est simple, mais la position et la vitesse sont **décalées** : on intègre la vitesse avec l'accélération **d'avant** le déplacement.

La loi de Hooke (rappel — la source du problème)

Un ressort exerce une force de rappel proportionnelle à l'étirement :

$$\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$$

- k : raideur, L : longueur actuelle, L_0 : longueur au repos, $\hat{u}$: direction unitaire.

Pour un tissu rigide, on veut un k **élevé**. Mais avec Euler, un grand k rend le système **instable** : l'énergie augmente à chaque pas jusqu'à l'explosion.

*Force de Hooke — ce qu'on *évitera* dans le TP.*

```
// Force de rappel appliquée à la particule `a` par le ressort (a,b)
function springForce(a, b, k, restLength) {
  const delta = b.clone().sub(a);
  const L = delta.length();
  const u = delta.normalize(); // direction unitaire hat(u)
  return u.multiplyScalar(-k * (L - restLength)); // -k * (L - L0) * u
}
```

Ce code fonctionne, mais pour l'utiliser avec Euler sans explosion, il faudrait un dt minuscule ou un k faible (tissu mou). C'est inacceptable pour un jeu à 60 FPS.

! La solution de cette session

Au lieu de **calculer des forces** (Hooke) puis d'intégrer (Euler), on va :

1. Intégrer avec **Verlet** — une méthode **symplectique** qui ne dérive pas en énergie.

2. Maintenir la rigidité avec **PBD** — on corrige directement les positions, sans aucune force ni raideur k .

Résultat : un tissu stable à n'importe quel dt , sans paramètre k à régler.

2. L'Intégration de Verlet

Idée de Verlet

Verlet ne stocke **pas** la vitesse explicitement. La vitesse est **déduite** de la différence entre la position actuelle et la position précédente :

$$v \approx \frac{x_n - x_{n-1}}{dt}$$

La formule d'intégration devient :

$$x_{n+1} = x_n + (x_n - x_{n-1}) + a \cdot dt^2$$

🔍 D'où vient la formule ?

Par développement de Taylor de $x(t)$ autour de t_n :

$$x(t_n + dt) = x_n + v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

$$x(t_n - dt) = x_n - v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

En additionnant les deux, les termes en v_n s'annulent :

$$x(t_n + dt) + x(t_n - dt) = 2x_n + a_n \cdot dt^2 + O(dt^4)$$

D'où la formule de Verlet :

$$x_{n+1} = 2x_n - x_{n-1} + a_n \cdot dt^2$$

qui est équivalente à $x_n + (x_n - x_{n-1}) + a \cdot dt^2$. Notez que l'erreur passe de $O(dt^3)$ (Euler) à $O(dt^4)$ — Verlet est **plus précise**.

Verlet vs Euler

Critère	Euler explicite	Verlet
Vitesse stockée	Oui (v_n)	Non (implicite)
Précision	$O(dt^3)$	$O(dt^4)$
Stabilité énergie	Mauvaise (dérive)	Bonne (symplectique)
Déplacement manuel	Casse la vitesse	Vitesse s'ajuste seule
Coût mémoire	x, v	x, x_{prev}

Tableau 5. – Verlet est plus précise, plus stable, et tolère qu'on déplace les particules à la main — exactement ce dont PBD a besoin.

❗ Pourquoi PBD exige Verlet

En PBD, on **déplace directement** les positions pour satisfaire les contraintes (section 4). Avec Euler, ce déplacement manuel n'affecte pas v — la vitesse devient incohérente avec la position, et le système « explose » à la frame suivante. Avec Verlet, la vitesse étant $x_n - x_{n-1}$, **toute correction de position met à jour automatiquement la vitesse**. C'est le mariage parfait.

Utilisations typiques de Verlet : cordes, tissus, chevelures, tas de billes, ragdolls.

2.A. Implémentation de Verlet

Une particule Verlet possède :

- position : position actuelle x_n .
- prevPosition : position précédente x_{n-1} .
- acceleration : accélération cumulée des forces (gravité, vent, ...).
- pinned : booléen, fixée au décor (mât, clou, ...).

Une étape de Verlet pour une particule. Voici **exactement** la fonction à coller dans le TP (Mission 1).

```
function verletIntegrate(p, dt) {
  if (p.pinned) return;           // fixée : ne bouge pas
  const temp = p.position.clone();
  const velocity = p.position.clone().sub(p.prevPosition);
  p.position
    .add(velocity.multiplyScalar(DAMPING)) // vitesse implicite + damping
    .addScaledVector(p.acceleration, dt * dt); // a·dt²
  p.prevPosition.copy(temp);
}
```

- temp sauvegarde x_n avant de l'écraser, pour devenir x_{n-1} au prochain pas.
- velocity est $x_n - x_{n-1}$ (sans diviser par dt — le dt est absorbé dans $a \cdot dt^2$).
- DAMPING (section 3) est un coefficient ≤ 1 appliqué à la vitesse.

3. Le Damping (Amortissement)

Force d'amortissement (forme classique)

On ajoute une force opposée à la vitesse pour dissiper l'énergie :

$$\vec{F}_{\text{damping}} = -b \cdot \vec{v}$$

Sans damping, un système isolé oscillerait indéfiniment (aucune perte).

Damping en Verlet — forme implicite

En Verlet, la vitesse est **implicite** : $\vec{v} \approx x_n - x_{n-1}$. Au lieu d'ajouter une force $-b \cdot \vec{v}$ puis de réintégrer, on **multiplie directement** la vitesse par un coefficient $d \leq 1$:

$$\vec{v}_{\text{damped}} = d \cdot (x_n - x_{n-1})$$

- $d = 1$: aucun amortissement (oscillations perpétuelles).
- $d < 1$: une fraction de la vitesse est perdue à chaque frame.
- $d = 0$: la particule n'a plus d'inertie (tombe comme un bloc).

C'est cette forme — **un simple coefficient** — que vous utiliserez dans le TP.

Damping en Verlet.

```
const DAMPING = 0.99;    // 1 = aucun amortissement, <1 = dissipation

// Dans la boucle d'intégration :
//   velocity = position - prevPosition
//   position += velocity * DAMPING    // <-- amortissement
```

Le `multiplyScalar(DAMPING)` dans `verletIntegrate` (section 2.A) applique exactement cela.

💡 Choix du coefficient

Pour un drapeau, $d \approx 0.99$ donne un flottement naturel (l'énergie du vent compense les pertes). Pour une corde qui doit s'arrêter, $d \approx 0.9$ à 0.95 . Trop bas (< 0.9), le mouvement devient « pâteux » et artificiel.

4. Position Based Dynamics (PBD)

Principe de PBD

Au lieu de calculer des forces complexes (Hooke) et de les intégrer, on **corrige directement les positions** des particules pour qu'elles respectent des contraintes géométriques : distance, angle, volume, etc.

Une **contrainte** est une règle du type « p_1 et p_2 doivent être à distance L_0 ». Si la règle est violée, on déplace les particules pour la restaurer.

Contrainte de distance — la mathématique

Soient deux particules p_1, p_2 à distance $L = \|p_2 - p_1\|$. On veut $L = L_0$. L'écart est $\Delta L = L - L_0$. On corrige chaque particule de la moitié de l'écart, le long de la direction unitaire $\hat{u} = \frac{p_2 - p_1}{L}$:

$$p_1 \rightarrow p_1 + \frac{1}{2}\Delta L \cdot \hat{u} \quad ; \quad p_2 \rightarrow p_2 - \frac{1}{2}\Delta L \cdot \hat{u}$$

Si une seule particule est mobile, elle absorbe **tout** l'écart (ΔL au lieu de $\frac{\Delta L}{2}$). Si les deux sont fixées, on ne corrige rien.

Algorithme d'une frame PBD

1. **Intégration** — appliquer Verlet à toutes les particules (gravité, vent, damping).
2. **Contraintes** — pour chaque contrainte, corriger les positions (fonction `satisfyConstraint`).
3. **Itérations** — répéter l'étape 2 N fois pour converger (voir ci-dessous).
4. **Rendu** — afficher les positions corrigées.

Contrainte de distance PBD. Voici **exactement** la fonction à coller dans le TP (Mission 2).

```
function satisfyConstraint(p1, p2, restLength) {
  const delta = p2.position.clone().sub(p1.position);
  const dist = delta.length();
  if (dist === 0) return;
  const diff = (dist - restLength) / dist;           // écart relatif
  const correction = delta.multiplyScalar(0.5 * diff);
  if (!p1.pinned) p1.position.add(correction);
  if (!p2.pinned) p2.position.sub(correction);
}
```

- delta est $p_2 - p_1$; dist est L ; diff est $\frac{\Delta L}{L}$ (écart **relatif**).
- correction = delta * 0.5 * diff vaut $\frac{1}{2}\Delta L \cdot \hat{u}$ — exactement la formule ci-dessus.
- Les gardes !pinned gèrent les particules fixées : si une seule est mobile, elle reçoit la correction complète (l'autre add/sub est sautée).

💡 Itérations = raideur (le point clé de PBD)

Une **seule** passe de contraintes ne restaure pas exactement L_0 : corriger une contrainte en déplace les voisines, qui ne sont plus à leur distance de repos. C'est un système **couplé**.

La solution : **itérer**. En répétant `satisfyConstraint` N fois par frame, les corrections se propagent et le système converge vers une solution où **toutes** les contraintes sont proches de L_0 .

- $N = 1$: tissu très élastique, la gravité l'étire.
- $N = 5$: tissu mou mais cohérent.
- $N = 15$ à 20 : tissu rigide, réaliste pour un drapeau.
- $N = 50$: quasi-rigide (corde tendue).

Pas de paramètre k : la raideur émerge du nombre d'itérations. C'est ce qui rend PBD si robuste — et si utilisé dans les moteurs modernes (PhysX, Bullet, Havok).

💡 Pourquoi PBD est inconditionnellement stable

On ne fait que **déplacer** des positions d'une quantité bornée ($\Delta \frac{L}{2}$). Il n'y a aucune force qui puisse diverger, aucun k trop grand, aucun dt critique. Le pire cas (N faible) donne un tissu mou — jamais une explosion. À comparer avec Hooke + Euler, où un k un peu trop grand fait tout sauter.

5. TP : Drapeau en Tissu (Verlet + PBD)

💡 Objectif du TP

Animer un drapeau accroché à un mât. Le tissu est une grille de particules liées par des contraintes de distance. Le fichier de départ `session08_cloth.html` fournit la scène Three.js, le mât, la géométrie du drapeau, le vent et la GUI — il ne reste qu'à compléter **deux fonctions** en copiant-collant le code des sections 2.A et 4.

Mise en place

- Ouvrir `session08_cloth.html` dans le navigateur (via Live Server).
- La scène contient un drapeau rouge accroché à un mât vertical.
- Au départ, les deux fonctions sont vides : le drapeau tombe comme un bloc (la gravité est appliquée mais aucune contrainte ne maintient le tissu). C'est normal.
- La GUI propose : intensité du vent, damping, itérations PBD, fixer/libérer la colonne gauche, reset, FPS.

Missions

Mission 1 — `verletIntegrate(p, dt)`

Copier le code de la section 2.A dans le corps de la fonction. Attention : la particule fixée (`p.pinned`) ne doit pas bouger. La vitesse implicite est `position - prevPosition`, multipliée par `DAMPING`, puis on ajoute `acceleration * dt²`.

Mission 2 — `satisfyConstraint(p1, p2, restLength)`

Copier le code de la section 4. Calculer `delta = p2.position - p1.position`, la distance, l'écart relatif, et corriger chaque particule de la moitié — sauf les particules `pinned`, qui ne bougent pas.

Mission 3 — Observer et régler

Une fois les deux fonctions en place :

- **Itérations PBD** à 1 : le drapeau s'étire et se déchire visuellement (contraintes molles).
- **Itérations PBD** à 15–20 : le drapeau reste rigide et ondule avec le vent.
- **Damping** à 1.0 : oscillations perpétuelles. À 0.95 : le drapeau se stabilise.
- **Vent** à 0 : le drapeau pend sous la gravité. À 3 : il flotte horizontalement.
- Décocher **Fixer colonne gauche** : le drapeau se détache du mât et tombe (les contraintes restent, plus rien ne le tient).

Scénarios à observer

- **Drapeau qui flotte** : avec vent ≈ 2 , damping ≈ 0.99 et 15 itérations, le drapeau ondule de façon réaliste. C'est exactement la technique utilisée pour les drapeaux et capes dans les jeux.
- **Tissu qui se déchire** : avec 1 itération, les contraintes ne convergent pas — la gravité étire le tissu. Cela montre **pourquoi** on itère en PBD (section 4).
- **Stabilité** : aucun k à régler, aucune explosion. Verlet + PBD est inconditionnellement stable — c'est toute la motivation de la session (section 1).

⚠ Pièges à éviter

- Ne pas oublier le `if (p.pinned) return;` dans `verletIntegrate` — sinon le mât s'envole avec le drapeau.
- Dans `satisfyConstraint`, tester `!p1.pinned` et `!p2.pinned` **séparément** : si les deux sont fixées, on ne corrige rien ; si une seule est fixée, l'autre absorbe toute la correction (le code de la section 4 gère ce cas).
- Garder le `if (dist === 0) return;` pour éviter une division par zéro lorsque deux particules se superposent.
- Ne pas modifier l'ordre $p1/p2$ du `delta` : la correction doit aller dans le bon sens (le code de la section 4 est déjà correct).

Bonus

Ajouter des contraintes **diagonales** dans `initCloth` (en plus des horizontales et verticales) avec `restLength = SPACING * Math.sqrt(2)`. Observer : le tissu résiste mieux au cisaillement et ne se déforme pas en losange. C'est ce qu'on appelle la **shear constraint** dans les simulateurs de tissu.

Session 9 : TP - Particules

Objectifs du TP

- Implémenter un système de particules simple.
- Utiliser Verlet ou Euler pour mettre à jour les particules.
- Créer des contraintes de distance pour former des structures (tissu, corde).
- Observer les cycles de vie naissance / update / mort.

Contexte

Un système de particules simule un grand nombre de petits objets : pluie, feu, fumée, étincelles. C'est le moment d'appliquer concrètement l'intégration vue précédemment.

1. Structure d'une Particule

Propriétés d'une particule

- Position, vitesse, accélération
- Durée de vie (lifetime)
- Couleur, taille
- Masse (optionnelle)

2. L'Émetteur

Emitter

- Génère des particules à un taux donné (ex: 1000/s).
- Définit position, direction, vitesse initiale.
- Peut être un point, un cercle, une sphère ou un cône.

3. Le Cycle de Vie

Cycle de vie

1. **Naissance** : création avec paramètres initiaux.
2. **Update** : application des forces (gravité, drag), mise à jour de la position.
3. **Rendu** : dessin sous forme de billboard (quad face caméra).
4. **Mort** : désactivation et recyclage quand lifetime ≤ 0 .

4. Interactions Physiques

- **Gravité** : $\vec{a} = \vec{g}$.
- **Drag** : $\vec{F} = -k \cdot \vec{v}$.
- **Collisions simplifiées** : rebond sur un plan, par exemple.

```
if (particle.position.y < 0) {
    particle.position.y = 0;
    particle.velocity.y *= -restitution; // rebond mou
}
```

5. Contraintes et Structures

💡 Objectif avancé

Relier des particules par des contraintes de distance pour créer un tissu ou une corde. Utiliser Verlet pour la stabilité et résoudre les contraintes en plusieurs itérations.

6. Défis

- Créer un émetteur de pluie.
- Créer un feu de camp avec des particules qui montent et s'estompent.
- Créer un drapeau (tissu) avec des contraintes de distance.

Session 10 : Introduction à la physique des rotations

Objectifs de la session

- Passer de la particule au corps rigide.
- Comprendre le couple, le moment d’inertie et la vitesse angulaire.
- Relier les équations linéaires et angulaires.
- Implémenter un corps rigide qui tourne sous l’effet d’une force.

💡 L'objectif du jour

Un corps rigide ne fait pas que se déplacer, il **pivote** autour de son centre de masse. Cette session introduit les outils pour modéliser la rotation.

1. Dictionnaire Linéaire vs Angulaire

Concept	Linéaire	Angulaire
Position	$\vec{r}$ (m)	Angle θ (rad)
Vitesse	$\vec{v}$ (m/s)	Vitesse angulaire ω (rad/s)
Résistance	Masse m	Moment d’inertie I
Cause	Force $\vec{F}$	Couple τ
Loi	$\vec{F} = m\vec{a}$	$\tau = I\alpha$

Tableau 6. – Analogie entre mouvement linéaire et rotation

2. Le Couple (Torque)

💡 Le Couple

Le couple dépend de **où** on pousse. En 3D, il est donné par un produit vectoriel :

$$\vec{\tau} = \vec{r} \times \vec{F}$$

En 2D, il se simplifie :

$$\tau = r_x F_y - r_y F_x$$

où $\vec{r}$ est le bras de levier depuis le centre de masse.

3. Le Moment d’Inertie

Moment d’inertie

Le moment d’inertie I représente la résistance d’un objet à la rotation. Il dépend de la répartition de la masse.

$$I = \sum_i m_i r_i^2$$

Cas courants :

- Anneau mince : $I = mR^2$
- Rectangle autour de son centre : $I = \frac{1}{12}m(w^2 + h^2)$

4. Vitesse d'un Point sur le Corps**Vitesse d'un point P**

Si un corps se déplace à $\vec{v}_G$ et tourne à ω , la vitesse d'un point P à la distance $\vec{r}$ du centre est :

$$\vec{v}_P = \vec{v}_G + (\omega \times \vec{r})$$

En 2D :

$$v_{P.x} = v_{G.x} - \omega \cdot r_y$$

$$v_{P.y} = v_{G.y} + \omega \cdot r_x$$

5. TP : Spinning Box**Objectif**

Implémenter un rectangle qui tourne quand on clique dessus avec la souris.

Étape 1 : Classe Rigidbody

Ajouter les propriétés :

- angle
- angularVelocity
- torque
- inertia (calculée une fois au démarrage)

Étape 2 : ApplyForce(force, worldPoint)

1. Calculer le bras de levier $\vec{r} = \text{worldPoint} - \text{center}$.
2. Ajouter la force linéaire : $\vec{F}_{\text{total}} += \vec{F}$.
3. Ajouter au torque : $\tau += r_x F_y - r_y F_x$.

Étape 3 : Intégrateur

1. $a = \frac{F}{m}$
2. $\alpha = \frac{\tau}{I}$
3. $v += a \cdot dt$
4. $\omega += \alpha \cdot dt$
5. $p += v \cdot dt$
6. $\theta += \omega \cdot dt$
7. Remettre F et tau à zéro.

Session 11 : Broad et Narrow phases – Part 2

Objectifs de la session

- Comprendre le rôle de la Narrow Phase.
- Apprendre SAT et GJK pour la détection précise de collision.
- Voir comment générer un contact et résoudre la collision.

Rappel

La Broad Phase a filtré les paires candidates. La Narrow Phase répond à la question : « **Ces deux objets se touchent-ils vraiment ?** »

1. La Narrow Phase : Méthodes de Résolution

Objectif

On a maintenant des paires proches. Il faut déterminer si elles se touchent vraiment, puis générer un contact (point, normale, profondeur).

2. SAT (Separating Axis Theorem)

Principe de SAT

Si on peut trouver un axe sur lequel les projections de deux objets ne se chevauchent pas, alors ils ne se touchent pas.

On teste les axes normaux des faces et les produits vectoriels des arêtes. Si un seul axe sépare les objets, il n'y a pas collision.

Utilisation : Boîtes vs boîtes, polyèdres convexes simples.

3. GJK (Gilbert-Johnson-Keerthi)

Principe de GJK

On travaille dans l'espace de Minkowski ($A - B$). On cherche si l'origine $(0, 0, 0)$ est contenue dans cette forme en construisant un simplex (tétraèdre en 3D).

Fonction de Support

GJK pose la question : « **Quel est ton point le plus extrême dans la direction $\vec{d}$?** »

$$S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

- Sphère : Centre + Rayon $\cdot \vec{d}$
- Cube : coin correspondant au signe de $\vec{d}$

Utilisation : Formes convexes quelconques (capsules, cônes, meshes convexes).

4. Génération du Contact et Résolution

Contact

Une fois la collision détectée, on a besoin de :

- Point de contact.
- Normale de collision.
- Profondeur de pénétration.

Résumé du Pipeline

1. **Update Position** : $P = P + V \cdot dt$
2. **Broad Phase** : voisinage -> paires candidates.
3. **Narrow Phase** : AABB, SAT/GJK -> collision OUI/NON.
4. **Contact Generation** : point, normale, profondeur.
5. **Resolution** : impulsions pour séparer les objets.

Session 12 : TP - Les planètes - Runge-Kutta

Objectifs du TP

- Simuler un système solaire avec la gravité universelle.
- Comprendre pourquoi Euler « dérape » sur les orbites.
- Implémenter Runge-Kutta 4 pour une intégration plus stable.
- Trouver la bonne vitesse initiale pour une orbite circulaire.

1. La Gravité Universelle

Loi de gravitation universelle

Newton a découvert que tous les objets s'attirent mutuellement.

$$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$$

- G : constante gravitationnelle (6.674×10^{-11}).
- r : distance entre les centres.
- $\hat{u}$: direction de l'attraction.

Pourquoi on ne ressent pas l'attraction entre deux personnes ? Parce que G est minuscule. Il faut une masse planétaire.

2. Le Système Solaire

Objectif du TP

1. Créer un soleil massif au centre.
2. Ajouter des planètes avec des vitesses initiales perpendiculaires au soleil.
3. Appliquer la gravité universelle entre chaque paire.
4. Observer les orbites.

Défi : trouver la vitesse initiale pour une orbite circulaire parfaite.

$$v = \sqrt{G \frac{M}{r}}$$

3. Pourquoi Euler ne suffit pas

Euler suppose que la vitesse est constante sur tout le pas de temps. Sur une orbite, la force change constamment, donc Euler accumule de l'erreur et les planètes « dérapent » ou s'échappent.

4. Runge-Kutta 4 (RK4)

Algorithme RK4

RK4 « sonde » le terrain à 4 endroits du pas de temps pour obtenir une pente moyenne.

1. $k_1 = \text{eval}(\text{état})$
2. $k_2 = \text{eval}(\text{état} + k_1 * h/2)$
3. $k_3 = \text{eval}(\text{état} + k_2 * h/2)$

$$4. \quad k_4 = \text{eval}(\text{état} + k_3 * h)$$

$$\text{Nouvel état} = \text{état} + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6} * h$$

Structure d'État

On regroupe position et vitesse dans un vecteur d'état Y .

$$Y = \begin{pmatrix} x \\ v \end{pmatrix}$$

$$\dot{Y} = \begin{pmatrix} v \\ a \end{pmatrix}$$

5. Missions

- Implémenter la gravité universelle entre N objets.
- Comparer Euler vs RK4 sur une orbite simple.
- Observer la stabilité des orbites sur plusieurs révolutions.
- Ajouter plusieurs planètes et voir leurs interactions.

Session 13 : Les moteurs physiques commerciaux

Objectifs de la session

- Comprendre pourquoi on utilise des moteurs physiques existants en production.
- Découvrir les moteurs majeurs et leurs cas d'usage.
- Introduire le déterminisme et l'architecture ECS.

1. Pourquoi utiliser un moteur physique ?

Avantages des moteurs existants

- **Complexité** : gestion des contacts multiples, stabilité, optimisation.
- **Temps** : des décennies de R&D.
- **Performance** : optimisé pour des milliers d'objets.

2. Concepts avancés

Rigid Body vs Soft Body

- **Rigid Body** : objet indéformable (boîte, sphère).
- **Soft Body** : objet déformable (tissu, gelée), beaucoup plus complexe.

Continuous Collision Detection (CCD)

Problème : un objet rapide peut traverser un mur entre deux frames (tunneling). **Solution** : on raycaste la trajectoire entre t et $t + \Delta t$ au lieu de tester seulement les positions.

Sleeping

Si un objet est immobile, on arrête de le simuler. Il « dort » et se réveille si un autre objet le touche.

3. Comparatif des moteurs

Moteur	Langage	Usage
PhysX	C++	AAA Games (Unity, Unreal default)
Havok	C++	AAA Games (Half-Life 2, Zelda TotK)
Jolt Physics	C++	Modern, open-source, Horizon Forbidden West
Box2D	C++	2D Games (Angry Birds, Terraria)
Cannon.js	JavaScript	Three.js (simple, legacy)
Ammo.js	JavaScript (WASM)	Three.js (Bullet port)
Rapier	Rust/WASM	Modern, fast, Three.js recommended

Tableau 7. – Comparatif des moteurs physiques

4. Déterminisme

Déterminisme

Un moteur est déterministe si les mêmes inputs produisent toujours les mêmes outputs.

Importance : multijoueur, replay, replays de gameplay.

Pièges : ordre des opérations, précision flottante selon le CPU.

5. Architecture ECS

Entity-Component-System

- **Entity :** un simple ID (ex: entity_42).
- **Component :** données pures (ex: Position, Rigidbody, Mesh).
- **System :** logique qui opère sur les composants (ex: PhysicsSystem, RenderSystem).

Avantage : flexibilité, on ajoute/enlève des composants à la volée.

Session 14 : Corps rigides – Boite de collision

Objectifs de la session

- Utiliser un moteur physique (Rapier.js) pour créer des corps rigides.
- Comprendre les types de Rigidbody.
- Découvrir les Colliders et leurs formes.
- Synchroniser le moteur physique avec Three.js.

🔗 Introduction

Maintenant que nous comprenons les concepts, nous allons utiliser un vrai moteur : **Rapier.js**. C'est un moteur 3D écrit en Rust, compilé en WebAssembly, et recommandé pour Three.js.

1. Les RigidBodies

Rigidbody

Un Rigidbody dans Rapier est l'objet principal. Il a :

- position, rotation, vitesse linéaire et angulaire.
- masse.
- un type : Dynamic, Fixed, Kinematic.

Types :

- **Dynamic** : réagit aux forces et collisions.
- **Fixed** : immobile, les autres rebondissent contre.
- **Kinematic** : contrôlé par le code, pousse les Dynamic.

Créer un Rigidbody dans Rapier.

```
import RAPIER from '@dimforge/rapier3d';

const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });

const rigidBodyDesc = RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(0, 5, 0);
const rigidBody = world.createRigidBody(rigidBodyDesc);

const colliderDesc = RAPIER.ColliderDesc.ball(1.0);
const collider = world.createCollider(colliderDesc, rigidBody);
```

2. Les Colliders

Collider

Le Collider définit la forme de l'objet pour la détection. Un Rigidbody peut avoir plusieurs Colliders.

Formes disponibles :

- ball(radius) : sphère.
- cuboid(hx, hy, hz) : boîte.
- capsule(halfHeight, radius) : capsule.

- `cylinder(halfHeight, radius)` : cylindre.
- `convexHull(points)` : forme convexe personnalisée.
- `trimesh(vertices, indices)` : mesh quelconque (statique).

3. Boucle de simulation

Synchronisation Three.js.

```
function animate() {  
    requestAnimationFrame(animate);  
  
    world.step();  
  
    for (const { mesh, rigidBody } of objects) {  
        const position = rigidBody.translation();  
        const rotation = rigidBody.rotation();  
        mesh.position.set(position.x, position.y, position.z);  
        mesh.quaternion.set(rotation.x, rotation.y, rotation.z, rotation.w);  
    }  
  
    renderer.render(scene, camera);  
}
```

Session 15 : Joints, Moteurs

Objectifs de la session

- Comprendre les types de joints et leurs cas d'usage.
- Savoir configurer des moteurs et des limites sur les joints.
- Découvrir les raycasts comme outil de requête dans le moteur physique.

1. Les Joints

Joints (Articulations)

Les joints relient deux RigidBody, limitant leur mouvement relatif.

Types :

- **Fixed Joint** : les deux corps sont rigidement liés (soudure).
- **Revolute Joint** : rotation autour d'un axe (charnière).
- **Prismatic Joint** : translation le long d'un axe (tiroir).
- **Spherical Joint** : rotation libre (épaule, rotule).

Configuration : chaque joint nécessite des **ancres** (anchors) locales sur chaque corps.

Créer une charnière dans Rapier.

```
const jointDesc = RAPIER.JointData.revolute(
  { x: 0, y: 1, z: 0 }, // ancre sur A
  { x: 0, y: -1, z: 0 }, // ancre sur B
  { x: 0, y: 1, z: 0 } // axe de rotation
);
const joint = world.createJoint(jointDesc, bodyA, bodyB);
```

2. Les Moteurs et Limites

Moteurs

On peut ajouter des moteurs aux joints pour les faire bouger activement.

```
joint.configureMotorVelocity(targetVelocity, stiffness, damping);
```

Limites

On ajoute des limites (min/max angle) pour empêcher une rotation excessive.

3. Les Raycasts

Raycast

Un raycast projette un rayon invisible et retourne le premier objet touché.

Utilisations : détection du sol, visée FPS, interaction, capteurs de véhicule.

Raycast dans Rapier.

```
const ray = new RAPIER.Ray(  
  { x: 0, y: 10, z: 0 },  
  { x: 0, y: -1, z: 0 }  
);  
const hit = world.castRay(ray, 20.0, true);  
if (hit) {  
  console.log("Distance:", hit.timeOfImpact);  
}
```

4. TP : Machine de Siège

Objectif

Construire une catapulte fonctionnelle :

1. Créer une base fixe.
2. Créer un bras avec un revolute joint.
3. Ajouter un moteur au joint.
4. Lancer un projectile.

Session 16 : Les contraintes – et l'IK

Objectifs de la session

- Comparer PBD et Impulse Based pour résoudre les contraintes.
- Comprendre l'algorithme PGS et les techniques de stabilité.
- Passer de la cinématique directe (FK) à l'inverse (IK).
- Découvrir CCD, FABRIK et leurs applications.

1. PBD vs Impulse Based

Deux approches pour résoudre les contraintes

A. Position Based Dynamics (PBD)

- Corrige directement les positions.
- Avantage : simple, stable visuellement.
- Inconvénient : la physique n'est pas « vraie » (vitesses incorrectes).
- Utilisé par : Verlet, PhysX (Soft Body), BeamNG.

B. Impulse Based

- Calcule les impulsions nécessaires pour respecter les contraintes.
- Avantage : physiquement correct, conserve la quantité de mouvement.
- Inconvénient : plus complexe, nécessite plusieurs itérations.
- Utilisé par : Box2D, Rapier, Havok, PhysX (Rigid Body).

2. Le Concept de Contrainte

Contrainte

Une contrainte est une équation qui doit être satisfaite à chaque frame.

Exemple : distance L entre deux particules A et B.

$$C = \|\overrightarrow{AB}\| - L = 0$$

Si $C \neq 0$, on applique une correction.

3. Algorithme PGS (Projected Gauss-Seidel)

PGS

Principe :

1. Pour chaque contact, calculer l'impulsion nécessaire.
2. L'appliquer.
3. Les vitesses ont changé, donc on recommence (itération).
4. Après N itérations, le système converge.

Typique : 4 à 8 itérations.

4. Techniques de Stabilité

Stabilité

Warm Starting : réutiliser les impulsions de la frame précédente pour converger plus vite.

Sleeping : les objets immobiles ne sont pas simulés.

Position Correction (Baumgarte) : corriger légèrement la position pour réduire la pénétration.

$$\text{correction} = \text{penetration} \cdot \text{facteur} \cdot \vec{n}$$

5. Forward vs Inverse Kinematics

FK vs IK

Forward Kinematics (FK) : on donne les angles et on calcule la position de la main.

$$\vec{p} = f(\theta_1, \theta_2, \dots)$$

Inverse Kinematics (IK) : on donne la position de la main et on calcule les angles.

$$\theta_1, \theta_2, \dots = f^{-1}(\vec{p})$$

Difficultés : plusieurs solutions possibles, parfois aucune solution, pas de solution analytique simple.

6. CCD (Cyclic Coordinate Descent)

CCD

On ajuste les articulations une par une, de l'extrémité vers la base.

1. Prendre la dernière articulation.
2. Calculer l'angle entre (articulation -> cible) et (articulation -> main).
3. Tourner l'articulation.
4. Passer à l'articulation précédente.
5. Répéter.

Avantage : simple, converge vite. **Inconvénient** : mouvements peu naturels.

7. FABRIK

FABRIK

On manipule directement les positions des articulations.

1. **Forward** : de la main vers l'épaule, en respectant les distances.
2. **Backward** : de l'épaule vers la main, en respectant les distances.
3. Répéter jusqu'à convergence.

Avantage : mouvements naturels, rapide. **Inconvénient** : ne gère pas les limites d'angles directement.

8. Applications de l'IK

- **Foot placement** : adapter les pieds au terrain.
- **Active ragdoll** : maintenir une pose via IK.
- **Hand grab** : attraper un objet précis.

Session 17 : TP - Véhicules

Objectifs du TP

- Comprendre le modèle « Raycast Vehicle ».
- Implémenter une suspension simple.
- Gérer la tenue de route (grip, slip angle).
- Créer un véhicule drivable avec Rapier.

1. Le Modèle Raycast Vehicle

Raycast Vehicle

Pour les jeux arcade, on ne simule pas les vraies roues. On utilise un raycast depuis le châssis vers le sol pour chaque roue.

Anatomie :

- Un châssis (RigidBody dynamique).
- Des roues invisibles : raycasts vers le bas.

Avantages : plus stable, plus performant, pas besoin de RigidBody pour les roues.

2. La Suspension

Suspension

Chaque roue simule un ressort.

1. Raycast depuis la roue vers le bas.
2. Si on touche le sol à distance d :
 - Compression : $c = 1 - \frac{d}{\text{maxSuspensionLength}}$
 - Force : $F = k \cdot c - \text{damping} \cdot v_{\text{suspension}}$
3. Appliquer la force au châssis.

3. La Tenue de Route

Grip et Slip Angle

- **Force longitudinale** : accélération / freinage.
- **Force latérale** : virage.
- **Slip angle** : angle entre la direction du pneu et la direction réelle du mouvement.
 - Petit angle : la voiture tourne normalement.
 - Grand angle : la voiture dérape (drift).

4. Missions du TP

1. Créer un châssis (boîte dynamique).
2. Ajouter 4 raycasts (roues).
3. Implémenter la suspension (ressort + damping).
4. Ajouter l'accélération (force longitudinale).
5. Ajouter la direction (force latérale sur les roues avant).

6. Tester sur un terrain off-road.

Défi

Trouver les bons paramètres de suspension pour que la voiture ne bascule pas dans les virages.

Session 18 : Personnages

Objectifs de la session

- Comparer Dynamic Character Controller (DCC) et Kinematic Character Controller (KCC).
- Implémenter Move and Slide.
- Gérer la détection du sol, le Coyote Time et le Jump Buffering.
- Créer un contrôleur FPS avec Rapier.

1. DCC vs KCC

Dynamic Character Controller (DCC)

Le personnage est un Rigidbody standard.

- **Avantages** : interagit naturellement avec le monde (poussé, tombe, etc.).
- **Inconvénients** : difficile à contrôler, peut glisser, être poussé.

Kinematic Character Controller (KCC)

Le personnage est un Rigidbody kinematic. On contrôle sa position manuellement.

- **Avantages** : contrôle total, game feel précis.
- **Inconvénients** : il faut coder les collisions, la gravité, les pentes.

La norme dans l'industrie : KCC pour les jeux AAA.

2. Move and Slide

Algorithme

1. Calculer le mouvement souhaité (input + gravité + vitesse).
2. Tenter de déplacer le personnage.
3. Si on touche un mur, glisser le long du mur.

Projection et Rejection

Soit $\vec{v}$ la vitesse et $\vec{n}$ la normale du mur.

- Projection (à annuler) : $\vec{v}_{\text{proj}} = (\vec{v} \cdot \vec{n})\vec{n}$
- Rejection (à conserver) : $\vec{v}_{\text{rej}} = \vec{v} - \vec{v}_{\text{proj}}$
- Nouvelle vitesse : $\vec{v}' = \vec{v}_{\text{rej}}$

3. Corner Trap

Quand on glisse le long de deux murs (un coin), le personnage peut se bloquer. Solution : traiter une collision à la fois, trier par profondeur, résoudre la plus profonde, puis re-tenter le mouvement.

4. Détection du Sol

Méthodes

- **Raycast simple** : un rayon depuis le centre. Simple mais peut rater les bords.

- **Shapecast** : on projette la capsule entière. Plus précis.
- **Multi-raycast** : plusieurs rayons sous les pieds.

Coyote Time et Jump Buffering

Coyote Time : après avoir quitté une plateforme, le joueur peut encore sauter pendant 0.1s.

Jump Buffering : si le joueur appuie sur saut juste avant d'atterrir, le saut s'exécute dès l'atterrissage.

5. TP : FPS Controller avec Rapier

1. Créer un Rigidbody kinematic (capsule).
2. Lire les inputs (WASD + souris).
3. Calculer la vitesse souhaitée.
4. Utiliser `world.castRay` pour détecter le sol.
5. Implémenter le Move and Slide.
6. Ajouter le Coyote Time et le Jump Buffering.

Session 19 : Physique, GPU et IA

Objectifs de la session

- Découvrir comment l'IA combine physique pour générer du mouvement.
- Comprendre Reinforcement Learning et Neuroevolution.
- Voir les capteurs physiques comme inputs d'un réseau de neurones.
- Introduire l'optimisation des particules avec le GPU.

1. Physique et IA : Mouvement Organique

Pourquoi mélanger IA et physique ?

Les animations pré-enregistrées sont rigides. La physique + l'IA permet de générer du mouvement en temps réel, adapté au terrain.

2. Reinforcement Learning (RL)

La boucle RL

- **Agent** : le personnage.
- **Environment** : le monde physique (Rapier).
- **State** : ce que l'agent voit (capteurs, positions, vitesses).
- **Action** : forces ou torques appliqués.
- **Reward** : note de l'action.

Exemple : apprendre à marcher. Reward = distance parcourue. Pénalité = tomber.

3. Neuroevolution (Genetic Algorithms)

Principe

1. Créer une population d'agents avec des réseaux de neurones aléatoires.
2. Les faire évoluer dans le monde.
3. Évaluer le fitness (distance, score).
4. Sélectionner les meilleurs.
5. Crossover et mutation.
6. Nouvelle génération.

4. Capteurs Physiques comme Inputs

Inputs du réseau

- **Raycasts** : N rayons vers l'avant, distance au premier obstacle.
- **Vitesse** : vitesse actuelle.
- **Orientation** : angle par rapport à la cible.

Exemple : 5 raycasts + vitesse + orientation = 7 inputs. 8 neurones cachés. 2 outputs (accélération + direction).

5. Optimisation GPU pour les Particules

Pourquoi le GPU ?

- Les particules sont indépendantes, idéales pour le parallélisme.
- Le GPU possède des milliers de cœurs.
- Les données restent sur le GPU (pas de transfert CPU/GPU).

Compute Shaders

- Programme qui s'exécute sur le GPU mais n'est pas un shader de rendu.
- Les particules sont stockées dans un SSBO (Shader Storage Buffer Object).
- Le compute shader met à jour positions, vitesses, lifetime.

Pipeline : CPU envoie les paramètres -> Compute Shader update -> Vertex/Fragment Shaders rendent.

6. TP : Voiture Autonome

Objectif

Créer une voiture qui apprend à naviguer un circuit :

1. Créer un circuit (murs, checkpoints).
2. Créer N voitures avec un Raycast Vehicle.
3. Chaque voiture a un réseau de neurones simple.
4. Inputs : 5 raycasts + vitesse.
5. Outputs : accélération + direction.
6. Fitness : distance parcourue avant de crasher.
7. Implémenter sélection, crossover, mutation.

Session 20 : Résumé - Quiz final

Objectifs de la session

- Réviser les concepts clés du cours.
- Faire le point sur les compétences acquises.
- Passer un quiz final pour valider les apprentissages.

Session présentielle

Révisions et Quiz (20 points).

Concepts clés à retenir

Fondamentaux

- Les trois lois de Newton.
- Les forces de la nature (gravity, normale, friction, drag).
- L'intégration numérique (Euler, Verlet, RK4).
- L'impulsion et les collisions.

Détection de collision

- Broad Phase (Grille, Quadtree, SAP).
- Narrow Phase (SAT, GJK).
- Génération de contacts et résolution.

Moteurs et systèmes avancés

- RigidBodies, Colliders, Joints, Moteurs.
- Contraintes et IK (CCD, FABRIK).
- Particules, GPU, personnages, véhicules, IA.

Quiz final

Format

Le quiz porte sur l'ensemble des sessions. Il mélange questions théoriques et petits problèmes à résoudre.